Artificial Intelligence

Algorithms and Applications with Python

Chapter 3



Dr. Dominik Jung
dominik.jung@jung-isec.de

python

Recap: Who Should Choose this Lecture

This lecture is aimed at two complementary audiences:

- Intermediate information systems / computer science students who want to get a general understanding of artificial intelligence (AI), understand how AI works, and learn new strategies for solving diverse AI problems.

- Students from other domains who are planning to use AI methods (e.g. machine learning) in their future and want to understand why it works the way it does.



If you are familiar with Python you can skip this chapter and continue in chapter 4.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python

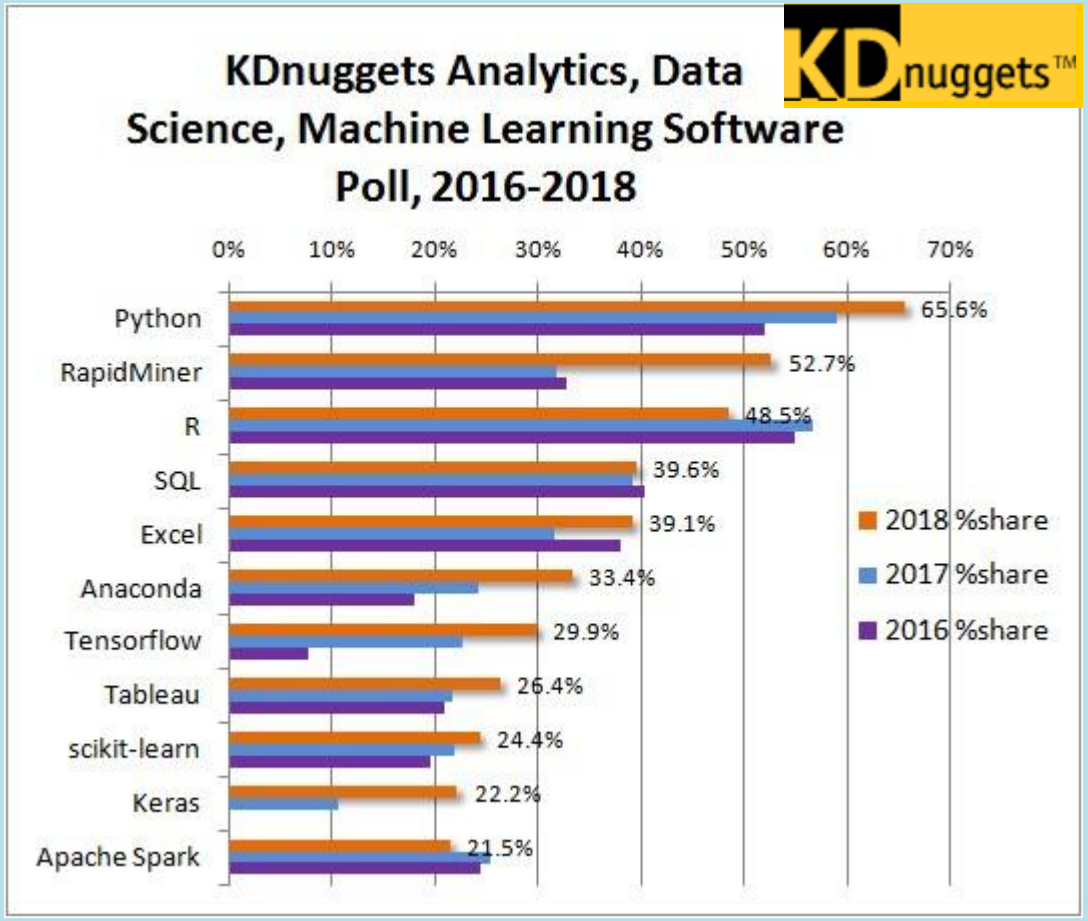


Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.1 Tools for AI Specialists



- The landscape of "Data Science Tools" is constantly changing
- Tools are often domain-specific: If you are interested in machine learning you better start with Python, if you are interested in statistics or analytics start with R
- But learn both!
- Tools are also customer-specific: If your customers can not program, you have to build your models with software tools

https://www.kdnuggets.com/2018/05/poll-tools-analytics-data-science-machine-learning-results.html

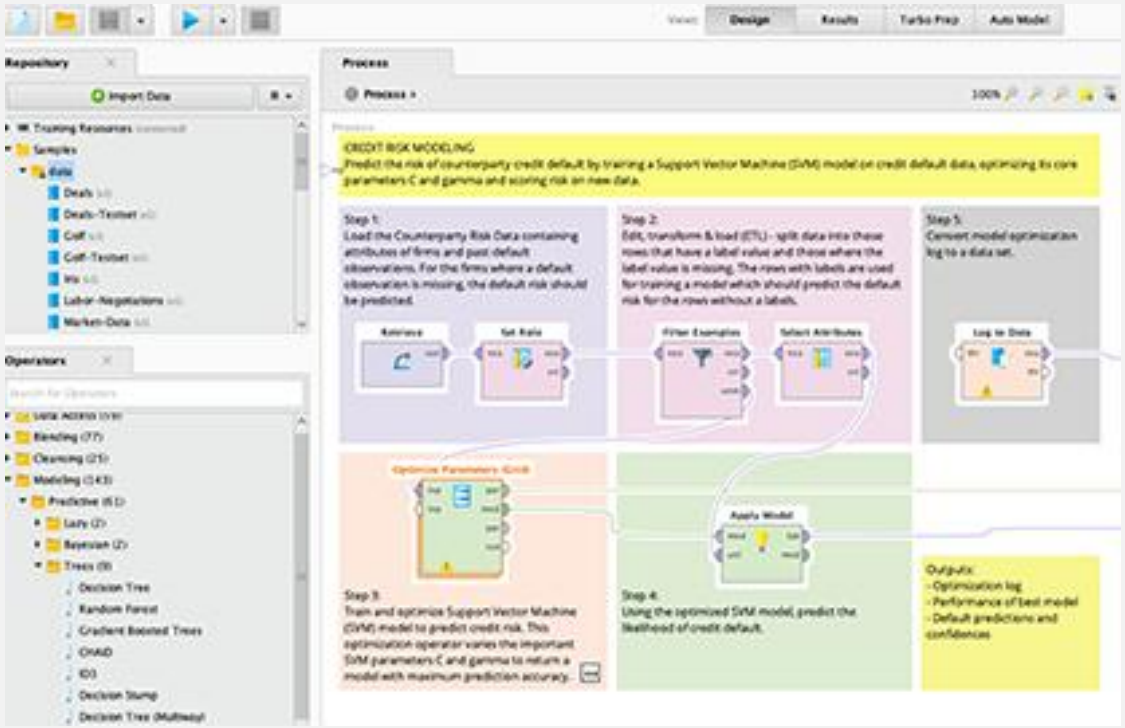
3.1 Machine Learning Tools: RapidMiner

Characteristics

- General purpose machine learning platform with a graphical user interface
- Developed from the AI Unit @TU Dortmund

Application

- Graphical workflow editor that supports all steps of the machine learning process (e.g. data processing, visualization and modelling)
- Many use cases for research, education, training, rapid prototyping, and application development



\+ Easy to use and no programming skills are necessary. Workflow can be communicated easily

\- Not as flexible and powerful as programming languages

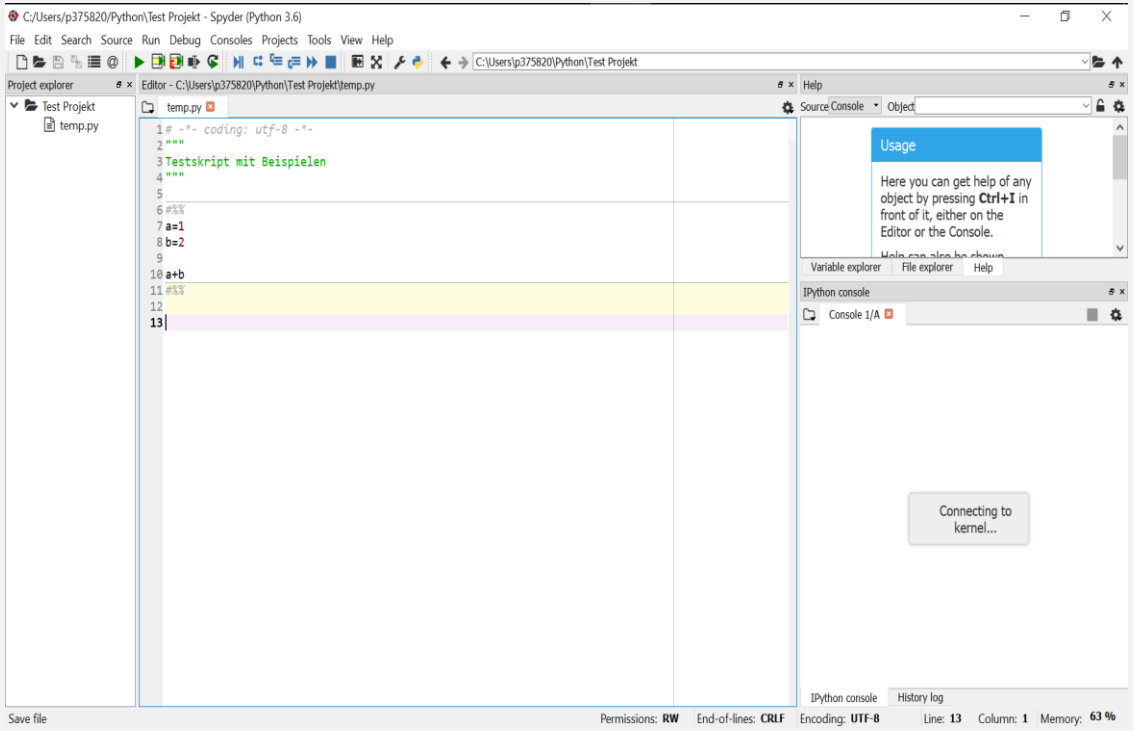
3.1 Programming Languages: Python

Characteristics

- General-purpose interpreted, interactive, object-oriented, and high-level programming language, widely used in various fields, thanks also to its readability
- Open source

Application

- Because it is a general purpose language, the use cases of Python are manifold
- Analytical/machine learning or data science P
- projects, where neural networks are used (e.g. for natural language processing)
- In particular, artificial neural networks (e.g. TensorFlow)



\+ Active and large community, with a huge number of great libraries for deep learning

\- Requires programming skills, but is easy to learn. Sometimes issues arising from version incompatibility

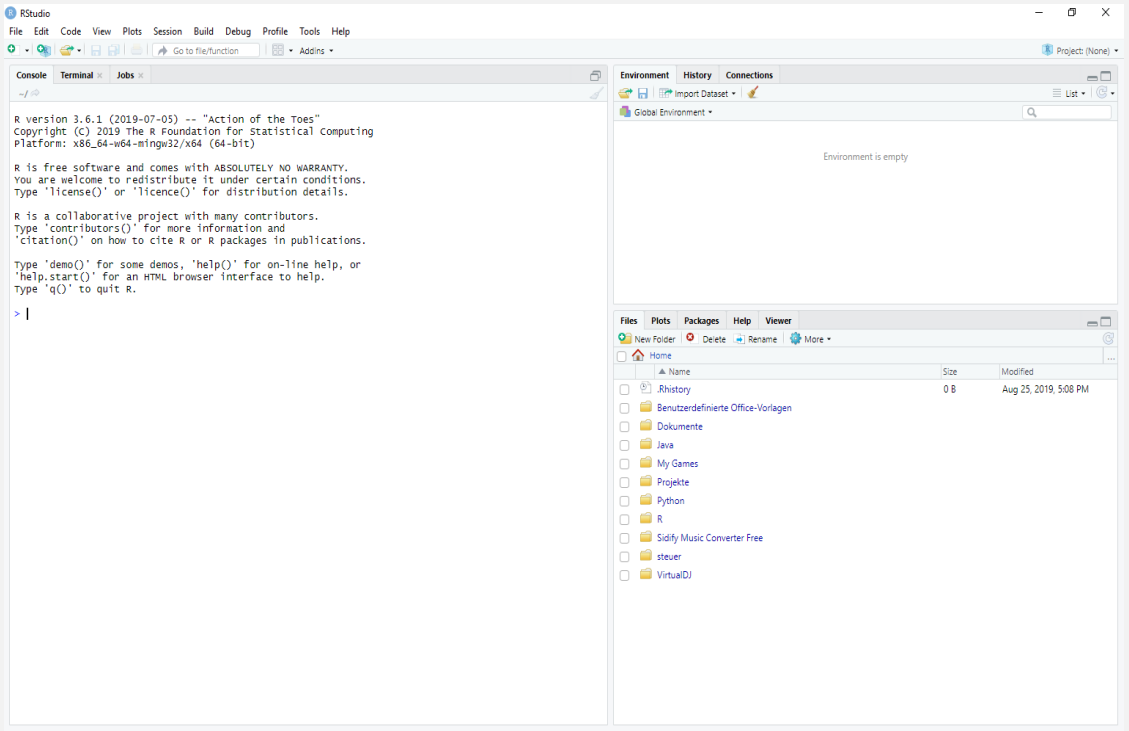
3.1 Programming Languages: R

Characteristics

- R: open source programming language designed for statistical computations
- State-of-the-Art in Analytics
- With R-Studio one of the best IDEs for Statistics

Application

- Popular among the academic and statistical community
- Excellent for Data Science Projects including statistical analysis (for small-to-medium sized amounts of Data) rather than Machine Learning



+ Large, active community with great libraries for visualization and dashboard design. Besides, language with the most statistical packages

- Slower and less readable than Python

3.1 Why do we use Python in this Course?

```
Public class Crawler{
Public static void main(String[] args) throws IOException{
    Print Writer textFile = null;
    try{
        textFile = new PrintWriter(„results.txt“);
        System.out.println(„Enter the URL you with to crawl…“);
        System.out.println(„URL: “);
        String myUrl = new Scanner(System.in).nextLine();

        String response = getContentByUrl(myUrl);

        Matcher matcher = Pattern
           .compile(„href=[\“‘](.[^\“‘)+)[\“‘]“).matcher(response);
        while(matcher.find()){
            String url = matcher.group(1);
            System.out.println(url);
            textFile.println(url);
        }
    finally{
        if(textFile != null){
             textFile.close();
        }
    }
}

Private static String getContentByUrl(String myUrl) throws IOException{
    Url url = new URL(myUrl);
    URLConnection urlConnection = url.openConnection();
…
```

```
If __name__ == ‘__main__‘:
    with open(„results.txt“,“wt“) as textFile:
        print(„Enter the URL you wish to crawl:“)
        myUrl = input(„URL: “)
        for i in re.findall(„href=[\“‘](.[^\“‘)+)[\“‘]“,
        urllib.request.urlopen(myUrl).read().decode(), re.I):
        print(i)
        textFile.write(i+‘\n‘)
```



Characteristics:

- Main purpose was to design an easy to learn language with strong focus on code readability
- Multi-purpose programming languages, thus very flexible and applicable to a broad range of cases
- Healthy, active and supportive community that develops state-of-the-art AI packages, e.g.:TensorFlow

Python

- Python has a couple of characteristics that have bolstered its rise in the data science community over the past years.
- One of the main aspects is its elegance and readability compared to other languages.

3.1 The Struggle is real

My team's reaction to using Power BI and realising they'll need to learn four or five programming languages (DAX, M, MDX, R etc) to deliver dashboards to users.



Software Tools

vs.

Programming Languages

3.1 What Is a Programming Language?

Programming Language
A formal language to write instructions for a computer. Like a human language it has a **syntax** (form: which statements are allowed) and a **semantics** (meaning: what a valid statement does).

Syntax

```
print("Hello!")
→ Hello!
print("Hello!"
→ SyntaxError: '(' was never closed
```

Python checks the syntax before it runs anything: one missing bracket and nothing runs at all.

Semantics

```
2 + 2
→ 4
"2" + "2"
→ '22'
```

Both lines are valid, but **+** means something different: it adds numbers and joins texts.

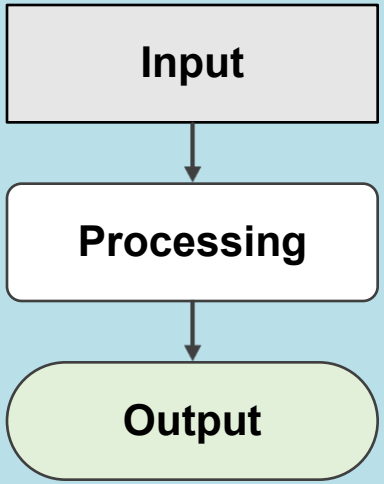
3.1 From Receipt to Program: The IPO Model

A Receipt

```
          CAMPUS CAFÉ
------------------------------
3 x Espresso    @ 1.10      3.30
1 x Croissant   @ 1.80      1.80
------------------------------
Subtotal                    5.10
Student discount 10 %      -0.51
==============================
TOTAL                   4.59 EUR
```

▶ Which parts of the receipt are data, which are rules, and which are results?

What the Program Does



Items, prices and quantities typed in at the till, plus the student card

Price × quantity for each line, add up the lines, subtract 10 % for students

The printed receipt with TOTAL 4.59 EUR

IPO Model

Every program reads **input**, **processes** it according to fixed rules and produces **output**. In German: EVA-Prinzip (Eingabe – Verarbeitung – Ausgabe).

3.1 Python in a Nutshell

Properties of Python

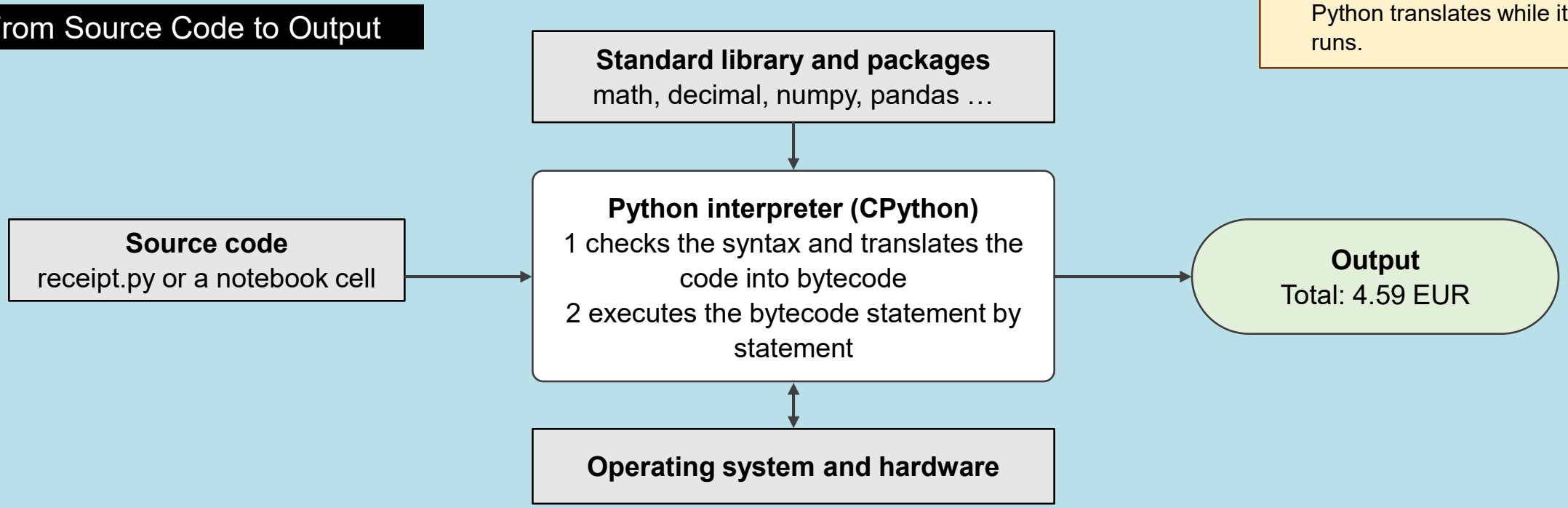
Property	What it means for you
High-level	Close to human language; Python handles memory and hardware details
Interpreted	An interpreter runs your code statement by statement
Dynamically typed	Types belong to values, not to variable names
Strongly typed	Python does not silently mix unrelated types
Multi-paradigm	Procedural, object-oriented and functional style are all possible
Batteries included	Large standard library plus a huge open-source ecosystem
Readable	Indentation instead of braces – „Readability counts.“ (Zen of Python)

▶ **History:** Python was created by Guido van Rossum and first released in 1991. Python 2 reached its end of life in 2020, so always use Python 3.

3.1 How Python Runs Your Code

Compiled languages such as C translate the whole program before it runs. Python translates while it runs.

From Source Code to Output



Interpreter
A program that reads source code and executes it directly, statement by statement, without first producing a separate machine-code program.

3.1 Three Ways to Run Python

Ways to Run Python

Mode	How it works	Best for
Interactive mode	Type python in a terminal; every line after >>> runs at once	Quick experiments and calculations
Script	Save the code as receipt.py and run python receipt.py	Programs you run again and again
Notebook	Code, results and text in cells, saved as .ipynb (Jupyter, Google Colab)	Data analysis, teaching, sharing results



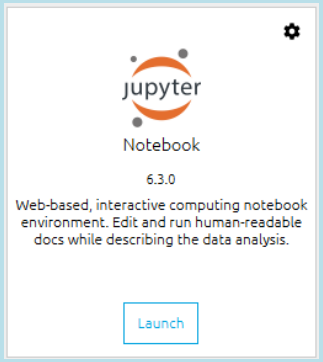
In this course we use **Google Colab**: it runs in your browser and needs no installation. ↗ https://colab.research.google.com



Want to work on your own computer? Install Anaconda (Jupyter, Spyder) or VS Code: ↗ https://www.anaconda.com/download – self-study, not relevant for the exam.

3.1 The AI Programming Toolbox - Popular Tools and IDEs for Python AI Programming

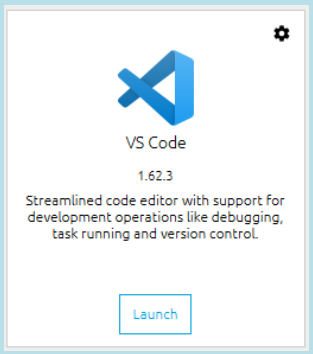
Applied AI & Data Science

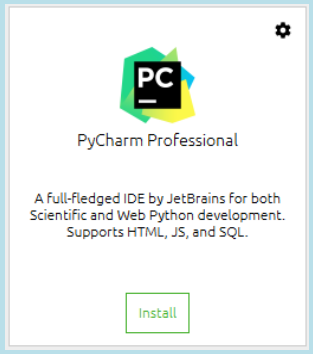




- Google CoLab, Jupyter Notebook and Jupyter Lab are tools to develop Python in your browser or on an AI-server
- Google CoLab and Jupyter Lab is a kind of mix of Jupyter Notebook and Spyder that runs in your browser

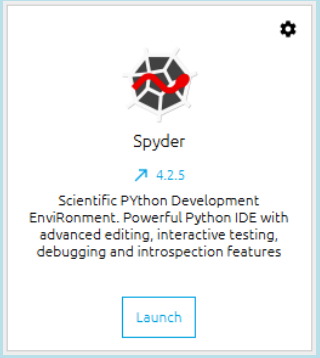
AI Development





- VS Code is my favorite IDE for web/software development and Python coding. It is free to use and supports many other programming languages
- PyCharm is an IDE that is also very popular in software development with Python

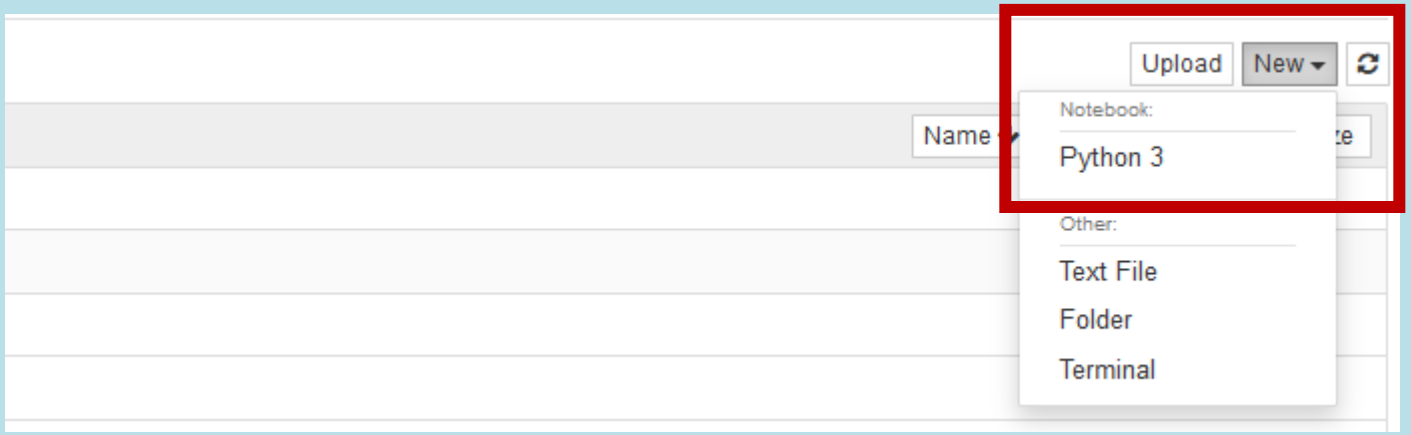
Data Analytics





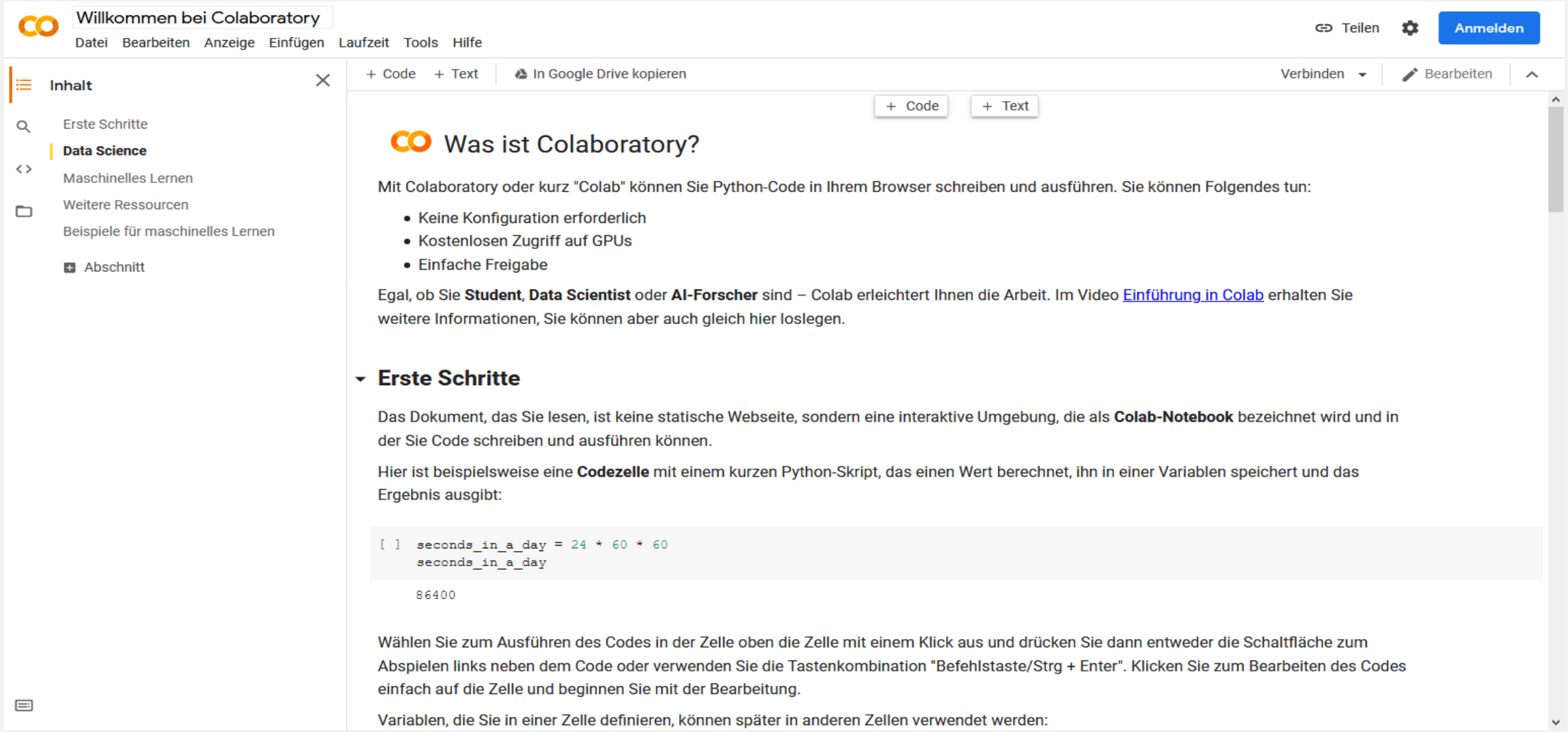
- Spyder is the most popular Python IDE for data analytics and diving deep in your data. In runs on your local machine
- Notebooks are best practice to share and discuss code with none-AI specialists or other experts due to their code buckets

3.1 Jupyter Notebooks

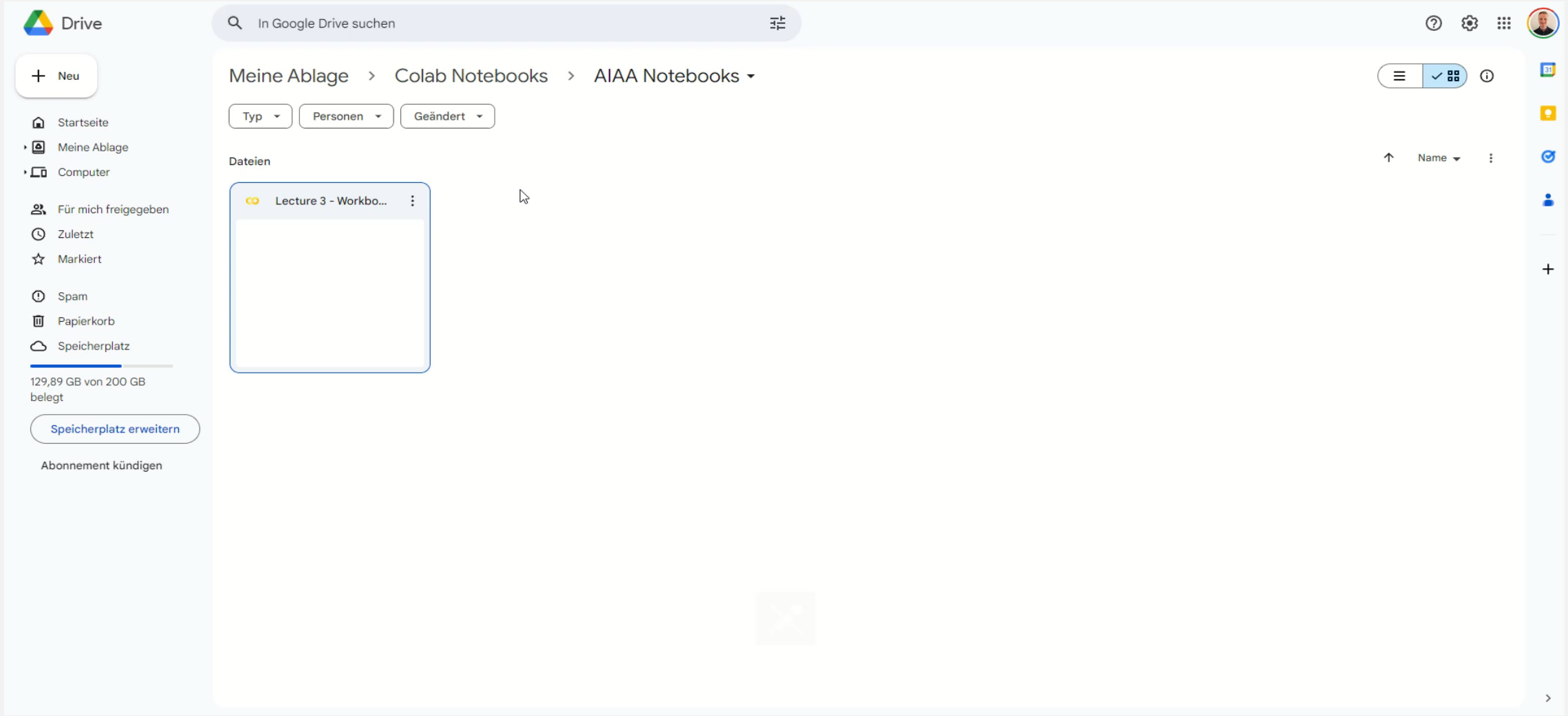


- Jupyter allows you to build simple code/text documents with your Python code

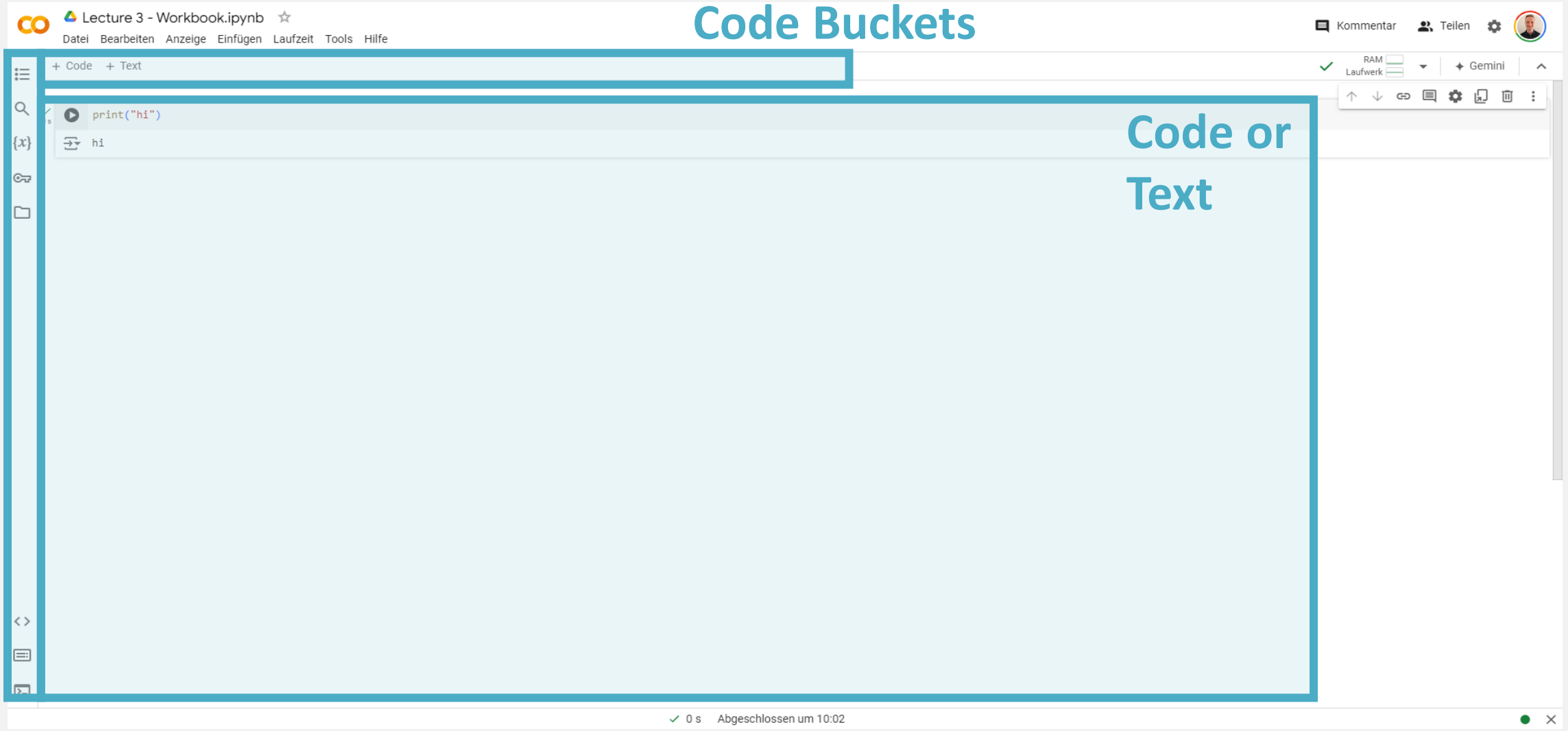
3.1 Jupyter Notebook in the Cloud: Colaboratory



3.1 Ready-to-go: Google Colabs

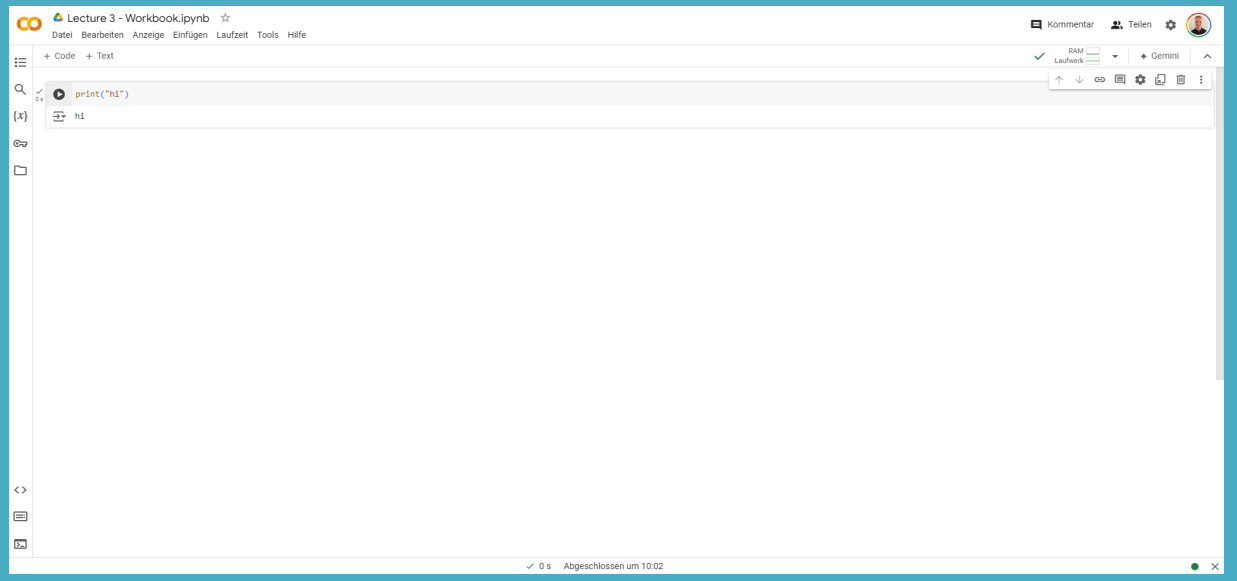


3.1 Google Colabs Overview

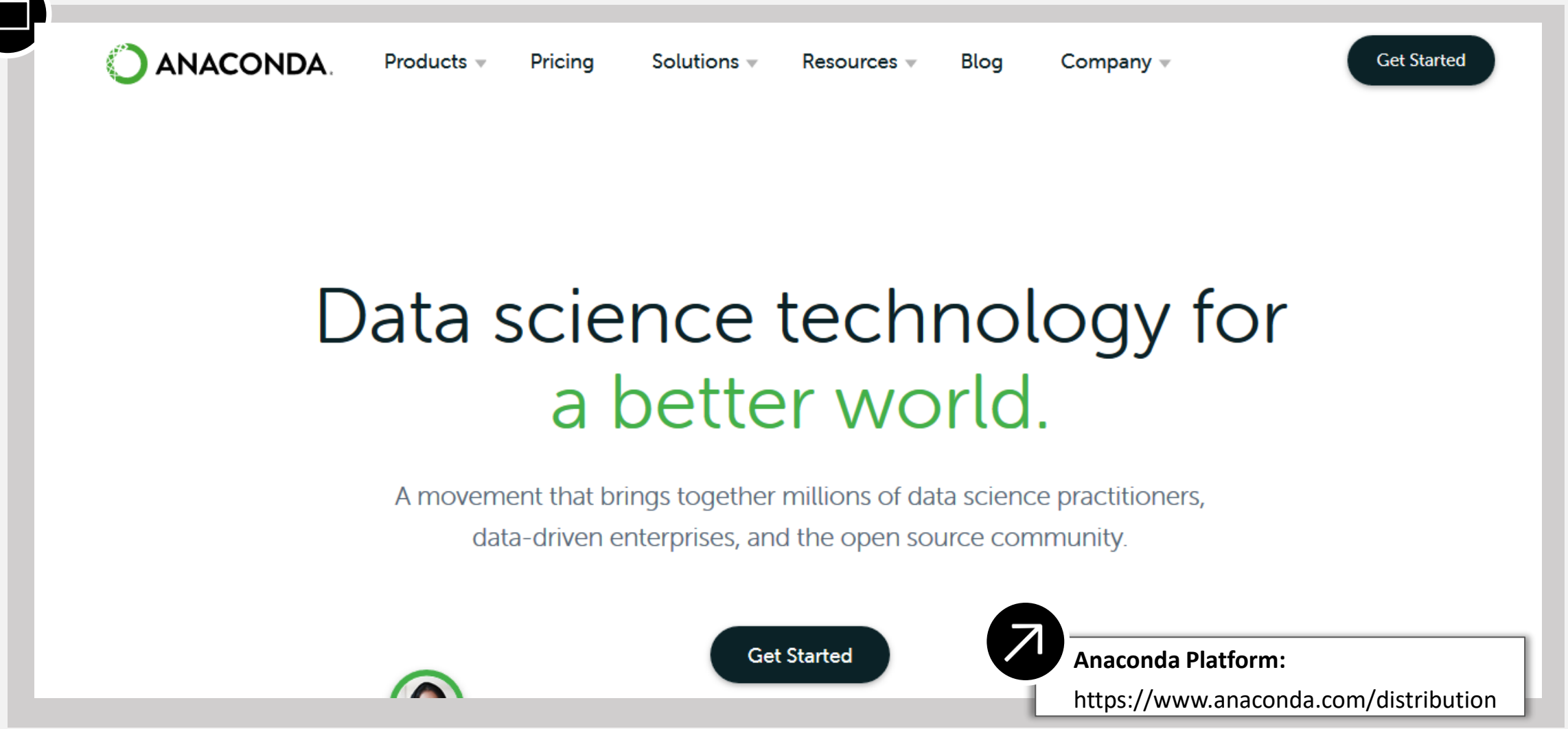


Now, please start Google CoLab in your Browser



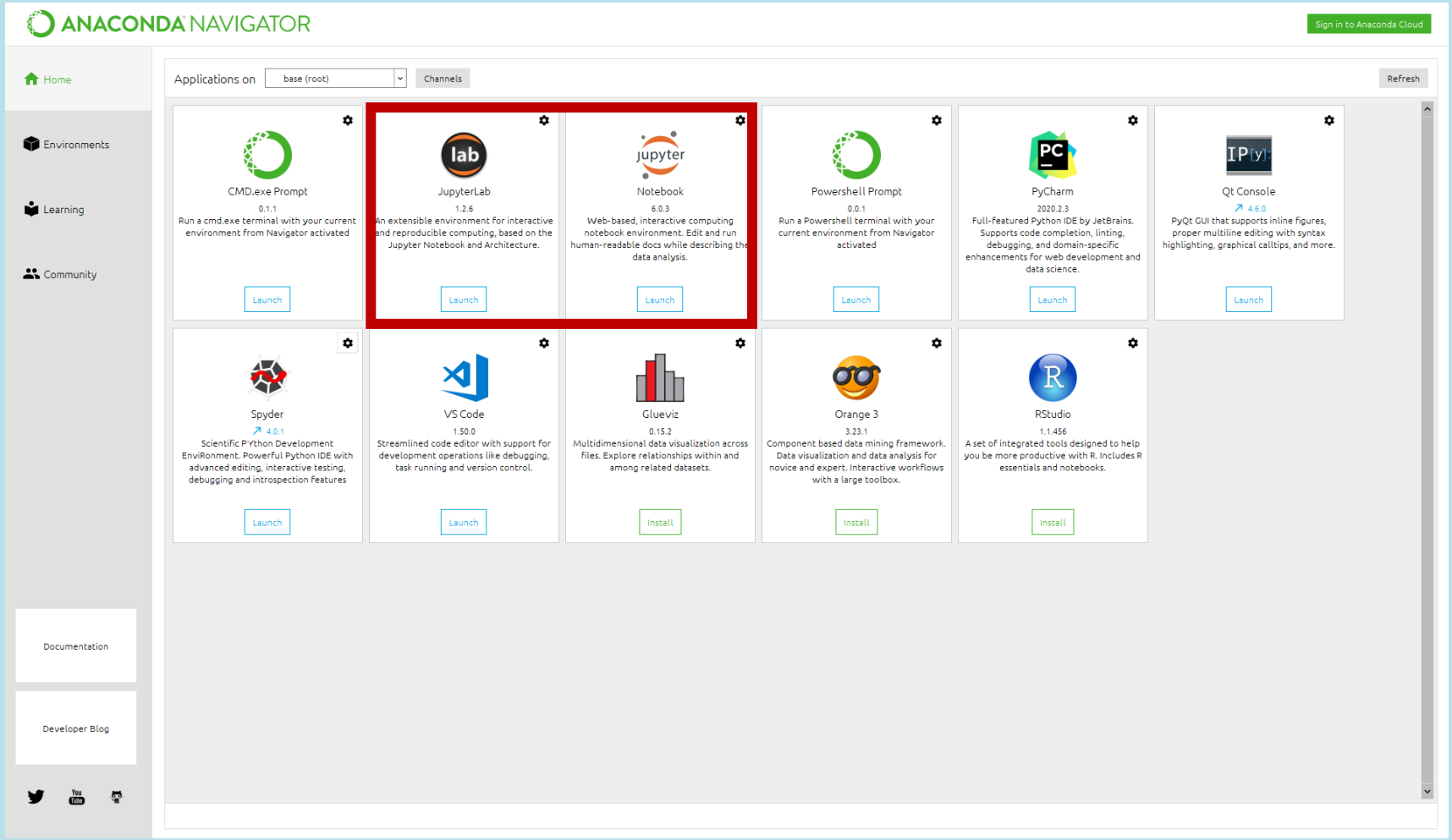


3.1 Install Anaconda

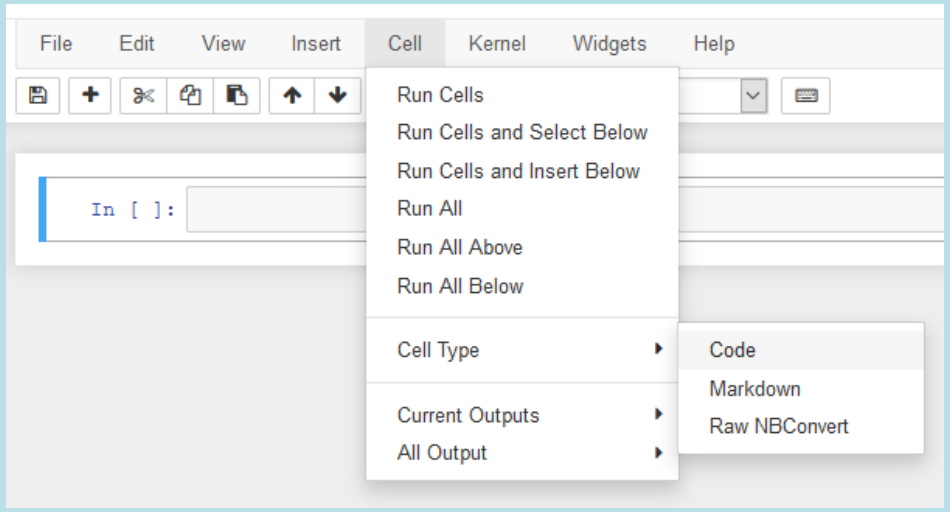


Anaconda Platform:
https://www.anaconda.com/distribution

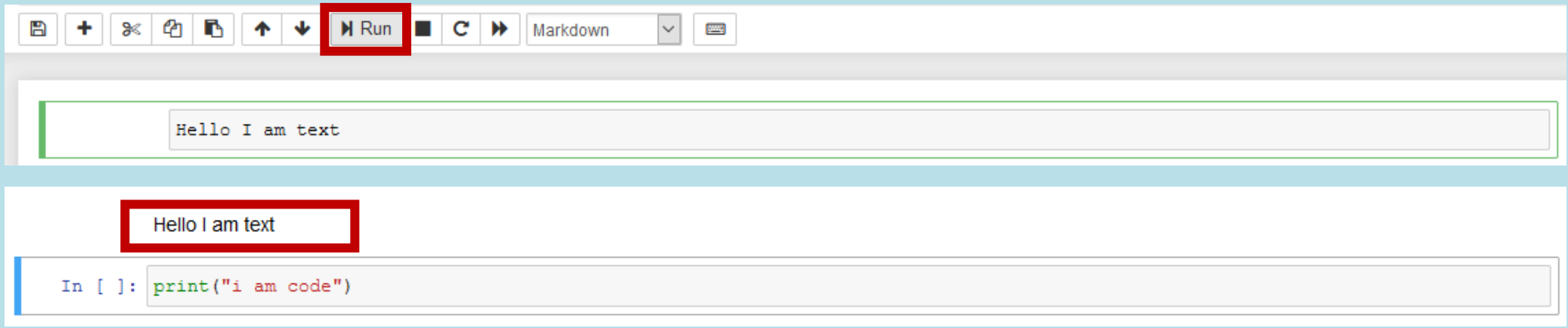
3.1 Start Jupyter from Anaconda



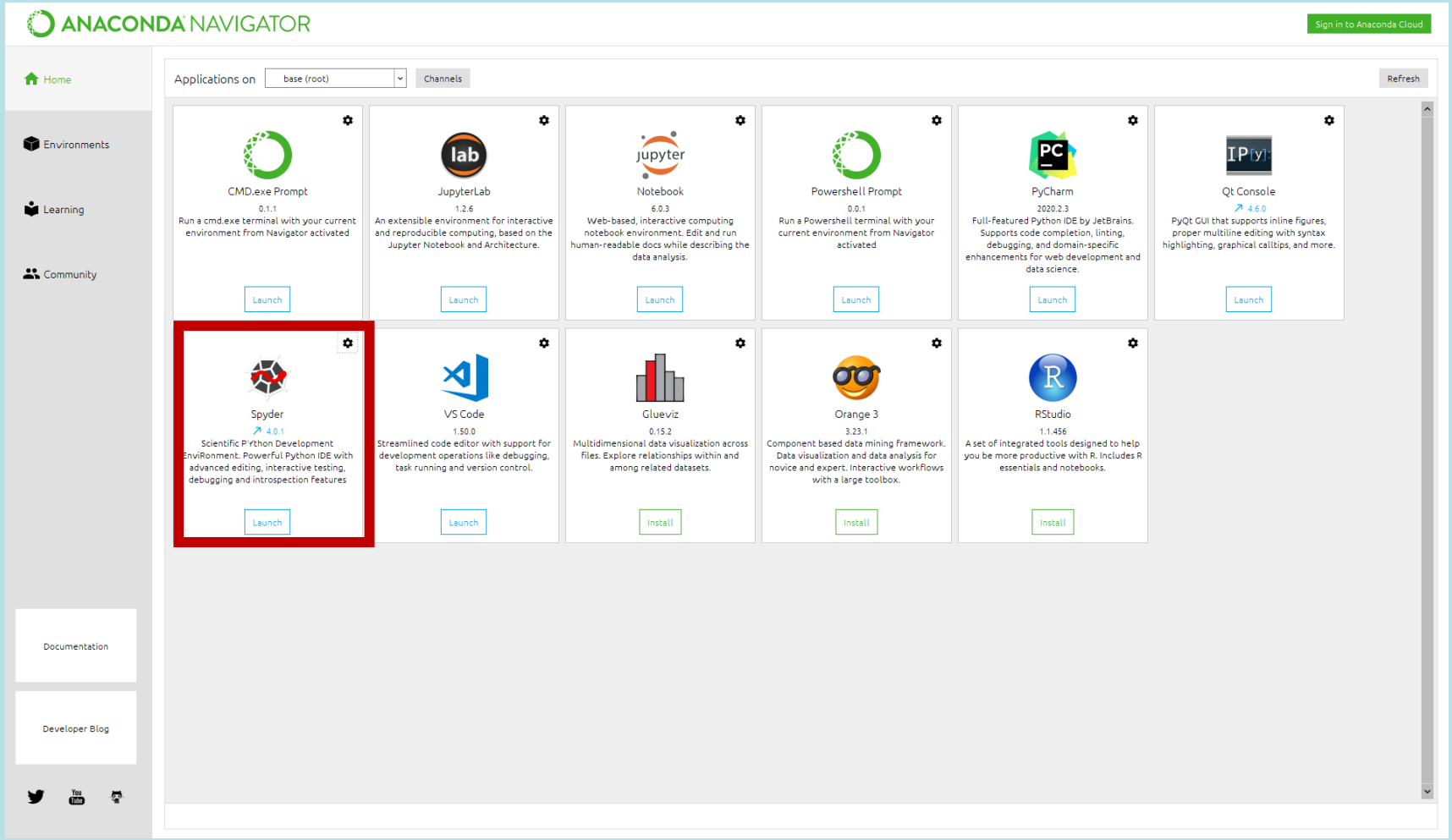
3.1 Jupyter Notebooks / Labs



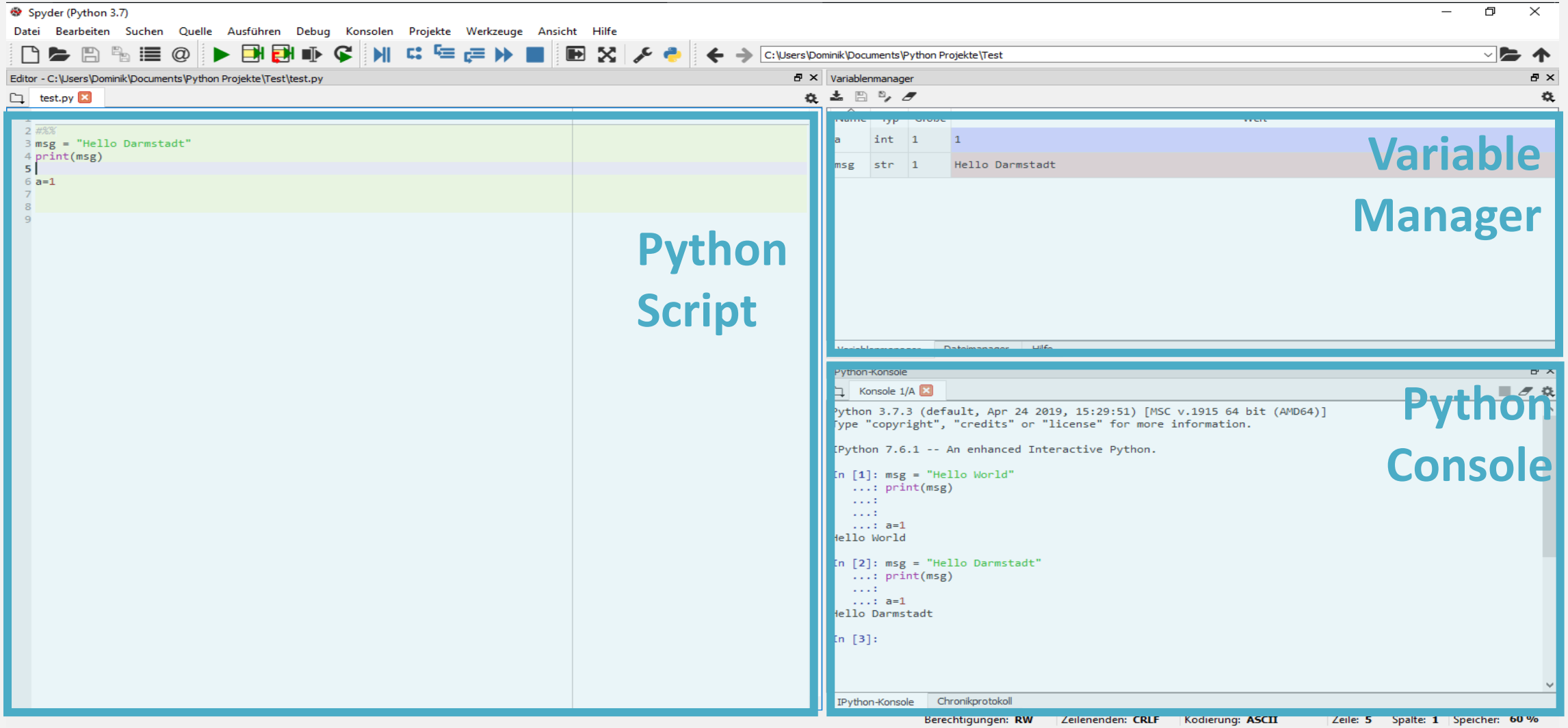
- Jupyter works with different cells. You can mark cells as text (markdown) or code (Python code) cells.
- If you press, run they get activated



3.1 Lets Take a Quick Look at Programming with the Spyder IDE in Anaconda

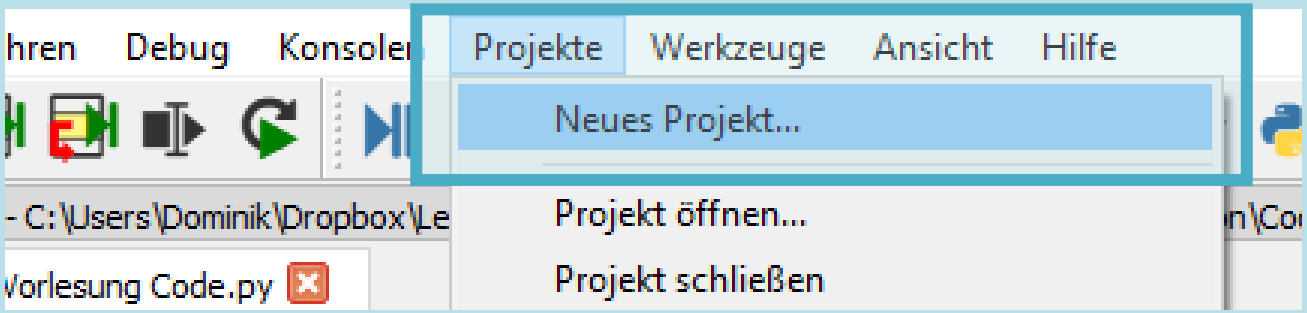


3.1 Spyder IDE

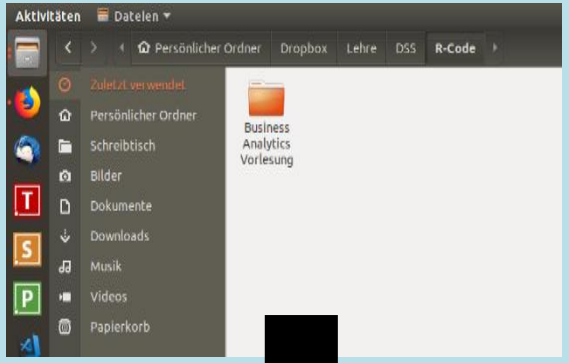


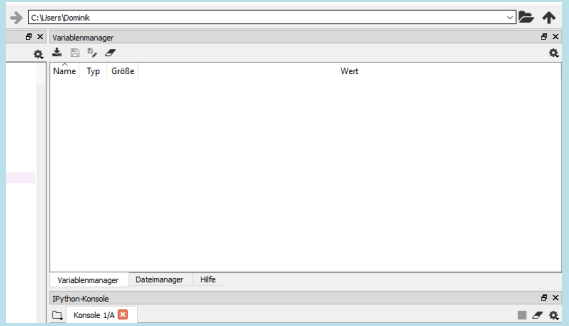
3.1 Projects with Spyder

- Spyder allows you to manage all your files and data with “projects”
- The benefit is that you can throw all your stuff in one folder and have direct access without browsing your files
- Furthermore you can save your current session
- And reload it easily



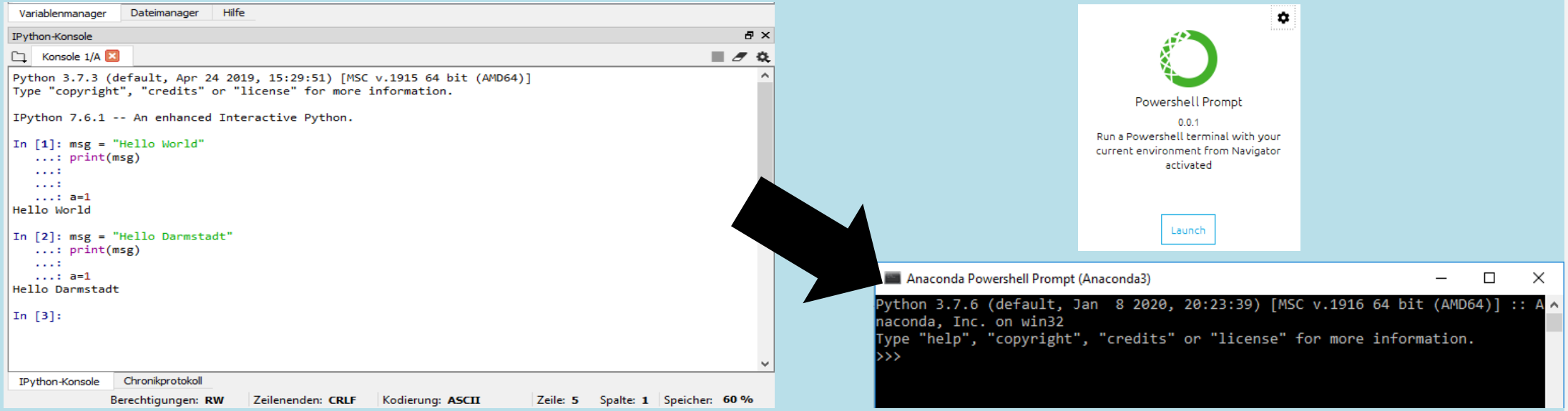
Data Management





3.1 First Steps in the Interactive Mode

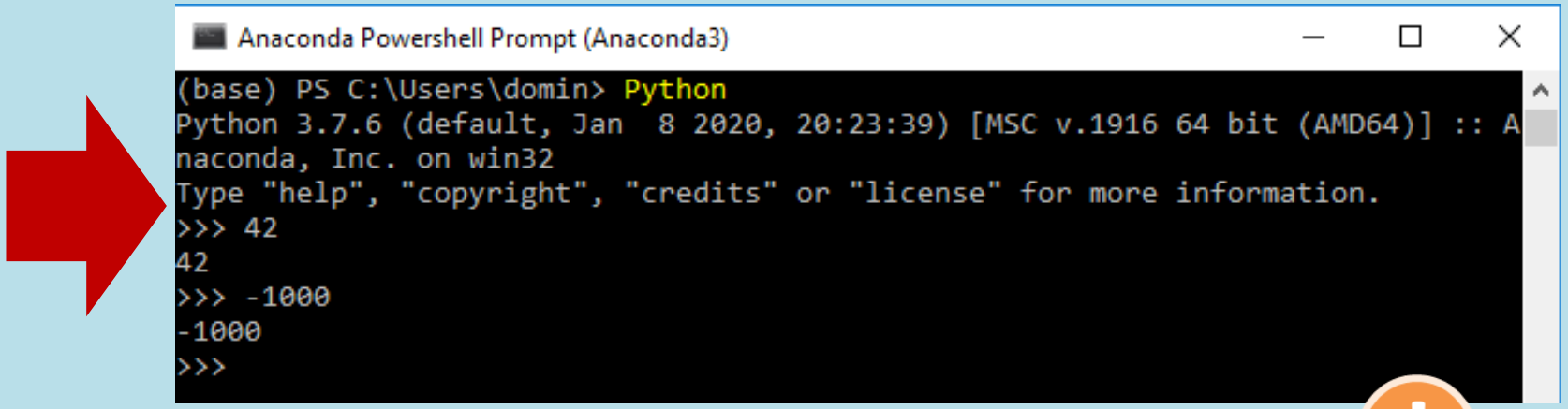
- To run Python commands you can use the console from the Spyder IDE or open the Anaconda-Shell and type „Python“



3.1 First Steps in the Interactive Mode

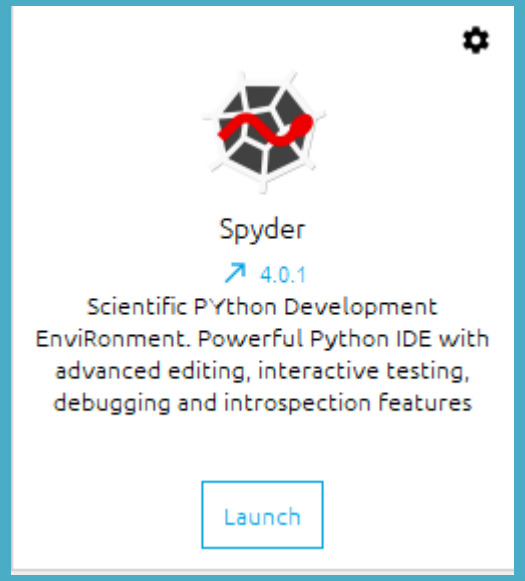
- Let us start by printing the following command in the Python interpreter (in the Spyder IDE or in the Python prompt directly)

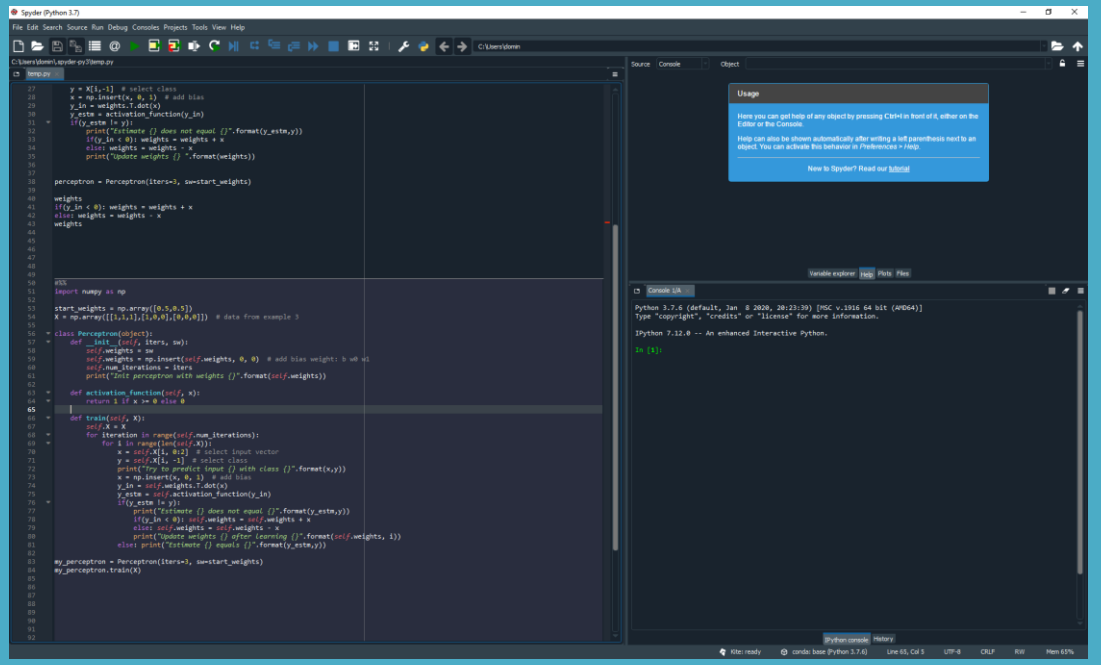
```
42
```



The interactive Python mode has a history functionality. Press ↑ or ↓ to re-read your previous code.

Now, please start the Spyder IDE from your Anaconda!





Your turn!

Task

Open Google Colab (↗ https://colab.research.google.com), create a new notebook and run your first line of code:

```
print("Hello, Campus Café!")
```

Task

Predict the result of the following line before you run it. Then run it in a new cell. What do you notice?

```
1.10 * 3
```

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.2 Expressions and Operators

Expressions

- An **expression** is code that Python evaluates to a value, e.g. 3 * 1.10 or "Espresso".
- Precedence as in math: ** first, then * / // %, then + and –. Parentheses make it explicit.
- / always returns a float; // and % turn 459 cents into 4 euros and 59 cents.

```
>>> 3 + 1 * 2
5
>>> (3 + 1) * 2
8
```

Arithmetic Operators

Operator	Meaning	Example	Result
+	addition	3 + 1	4
-	subtraction	10 - 4	6
*	multiplication	3 * 2	6
/	division	459 / 100	4.59
//	floor division	459 // 100	4
%	remainder	459 % 100	59
**	power	2 ** 3	8

3.2 Variables

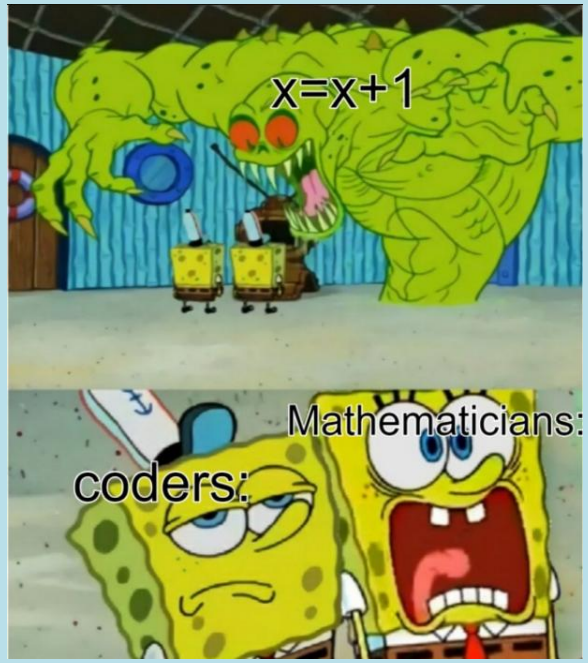
- Consequently, you can define variables

```
>>> name = 0.5
>>> var123 = 12
>>> string = "Hallo Welt!"
>>> liste = [1,2,3]
```

- You can access variables later by „calling“ them

```
>>> name
0.5
>>> 2 * name
1.0

>>> else = 4
?
```



Error to run `else = 4`? Some variable names are not allowed in Python, they are reserved for other purposes (e.g. “and”, “if”, “else” etc)

Adapted from Ernesti, J. & Kaiser, P. (2017)

3.2 Variables Are Names for Values

Assignment

```
>>> price = 1.10
>>> quantity = 3
>>> subtotal = price * quantity
>>> quantity = quantity + 1
>>> quantity
4
```

- A **variable** is a name that refers to a value. You create it with an assignment: name = expression.
- = is not an equation: Python evaluates the right side first, then binds the name on the left. That is why quantity = quantity + 1 works.

Rules for Names

- Letters, digits and underscores; no digit at the start
- Case-sensitive: Price and price are two different names
- Not one of the 35 keywords such as if, else, for or True
- Style guide PEP 8: lowercase words joined by underscores (snake_case), e.g. unit_price, is_student

```
>>> else = 4
SyntaxError: invalid syntax
```

3.2 Basic Data Types

Basic Data Types

Type	Kind of value	On our receipt	Example
int	whole numbers	quantity	3
float	decimal numbers (floats)	price	1.10
str	text (string)	item name	"Espresso"
bool	truth values	is the customer a student?	True, False
NoneType	no value	discount not known yet	None

```
>>> type(1.10)
<class 'float'>
>>> type("Espresso")
<class 'str'>
```



True, **False** and **None** start with a capital letter. TRUE or true raise a NameError.

3.2 Dynamic and Strong Typing

Dynamic Typing

- The type belongs to the object, not to the name.
- A name can refer to objects of different types over time; Python checks types while the program runs.

```
>>> x = 3
>>> type(x)
<class 'int'>
>>> x = "three"
>>> type(x)
<class 'str'>
```

- Statically typed languages such as Java fix the type of a variable: int quantity = 3;

Strong Typing

- Python does not silently convert between unrelated types.

```
>>> "3" + 3
TypeError: can only concatenate
str (not "int") to str
>>> int("3") + 3
6
>>> "3" * 3
'333'
```

- Weakly typed JavaScript would return "33" for "3" + 3.
- Numbers still mix: 3 + 1.5 gives 4.5.

3.2 Why 1.10 × 3 Is Not Exactly 3.30

The Problem

```
>>> 1.10 * 3
3.3000000000000003
>>> 0.1 + 0.2 == 0.3
False
```

- Floats are stored in binary with 64 bits (IEEE 754).
- Most decimal fractions such as 0.1 have no exact binary form – just as 1/3 has no exact decimal form (0.333…).
- The tiny rounding errors show up in results and comparisons.

What You Can Do

```
>>> round(1.10 * 3, 2)

3.3

>>> f"{1.10 * 3:.2f}"

'3.30'

>>> 110 * 3       # cents

330

>>> from decimal import Decimal

>>> Decimal("1.10") * 3

Decimal('3.30')
```

- Round only for output; compare floats with math.isclose(a, b), never with ==.
- For money, calculate in cents (int) or use Decimal.

Adapted from Goldberg, D. (1991) and the Python documentation, „Floating-Point Arithmetic: Issues and Limitations“

Your turn!

Task

Predict the result of each expression, then check it in Colab:

```
type(7 / 7)
7 // 2
"ab" * 3
0.1 + 0.2 == 0.3
int("3") + 3
```

Task

Two cappuccinos cost 2.40 EUR each. Store price and quantity in variables, compute the subtotal and print it with two decimals.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.3 Reading User Input

input() and print()

- input(prompt) shows the prompt, waits until the user presses Enter and returns the typed text – always as a **string**.
- print() writes values to the screen, separated by spaces.

```
>>> quantity = input("How many espressos? ")
How many espressos? 3
>>> quantity * 2
'33'
>>> int(quantity) * 2
6
>>> print("Quantity:", quantity)
Quantity: 3
```

- Convert the text before you calculate with it: int(), float().

A Classic Bug



```
>>> price = input("Price? ")
Price? 1.10
>>> price * 3
'1.101.101.10'
>>> price + 1
TypeError: can only
concatenate str
(not "int") to str
```

- * repeats the text, + refuses to add a number.
- Fix: float(price) * 3

3.3 Converting Between Types

Type Conversion

Call	Result	What happens
int("3")	3	text → whole number
float("1.10")	1.1	text → decimal number
str(4.59)	'4.59'	number → text
int(4.99)	4	cuts off the decimals, no rounding
round(4.59)	5	rounds to the nearest whole number
int("1.10")	ValueError	the text is not a whole number
float("1,10")	ValueError	a decimal comma is not allowed



German data often uses a decimal comma. Replace it before converting: float("1,10".replace(",", ".")) gives 1.1.

3.3 Strings and Formatted Output

Strings

- A string is a sequence of characters in quotes. It is immutable: you can read it, but not change it.
- Indexing starts at 0. Methods are functions that belong to an object: item.upper()

```
>>> item = "Espresso"
>>> len(item)
8
>>> item[0]
'E'
>>> item.upper()
'ESPRESSO'
>>> "  latte \n".strip()
'latte'
```

f-Strings

- Put an f before the quotes and write expressions in braces.
- {total:.2f} shows two decimals; < and > align text in a fixed width.

```
>>> item = "Espresso"
>>> total = 3 * 1.10
>>> f"{item}: {total:.2f} EUR"
'Espresso: 3.30 EUR'
>>> f"{item:<12}{total:>8.2f}"
'Espresso        3.30'
```

Your turn!

Task

Write a program that asks for an item and a quantity, computes the line total with a price of 1.10 EUR and prints a receipt line like this one:

```
3 x Espresso        3.30 EUR
```

Task

Bonus: What happens if the user types three instead of 3? Read the last line of the error message.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



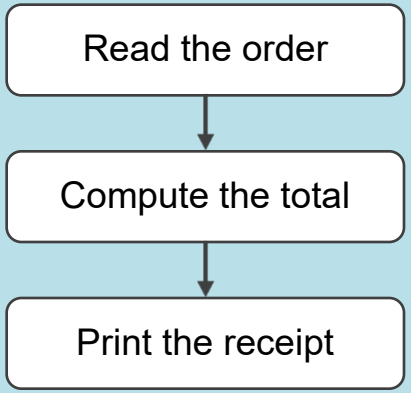
Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

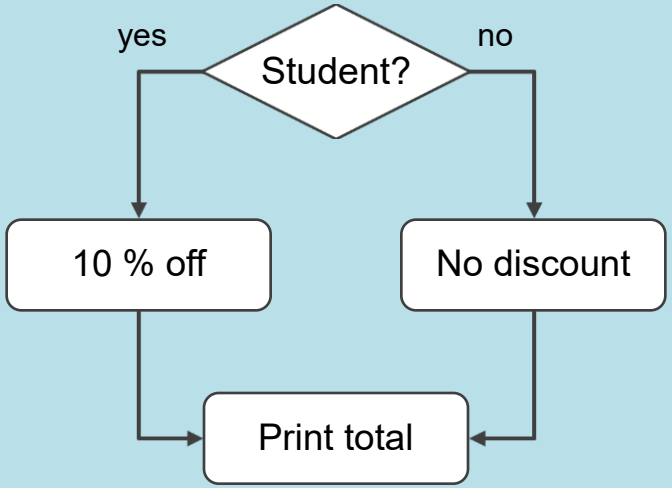
3.4 Three Building Blocks of Every Program

Sequence



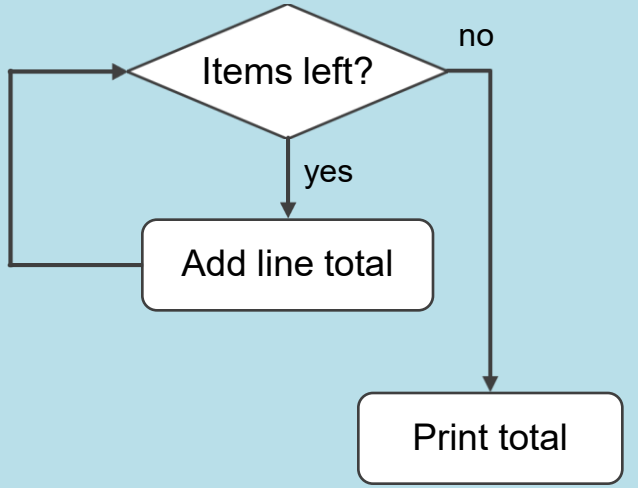
Statements run one after another, top to bottom.

Selection



Exactly one path is taken.

Iteration



for and while repeat a block (Part 2).

Structured Program Theorem (Böhm & Jacopini, 1966)
Every computable procedure can be built from three control structures alone: **sequence**, **selection** and **iteration**.

3.4 Comparisons and Boolean Logic

Comparisons in Python

```
>>> quantity = 3
>>> subtotal = 5.10
>>> is_student = True
>>> quantity == 3
True
>>> subtotal < 5
False
>>> is_student and subtotal >= 5
True
>>> not is_student
False
```



= assigns a value, **==** compares two values. Mixing them up in an if condition is a SyntaxError.

Comparison and Logical Operators

Operator	Meaning
==	equal to
!=	not equal to
< <=	less than (or equal to)
> >=	greater than (or equal to)
and	True if both sides are True
or	True if at least one side is True
not	turns True into False and vice versa

Every comparison returns a **bool**: True or False.

3.4 Branching with if, elif and else

Choosing a Discount

```
if is_student:
    discount = 0.10
elif subtotal >= 20:
    discount = 0.05
else:
    discount = 0.0


total = subtotal * (1 - discount)
print(f"Total: {total:.2f} EUR")
```

- Python checks the conditions from top to bottom and runs only the **first** block whose condition is True.
- elif and else are optional.

Which Discount Applies?

is_student	subtotal	discount	total
True	5.10	0.10	4.59
False	25.00	0.05	23.75
False	5.10	0.0	5.10
True	25.00	0.10	22.50



Why does a student with 25.00 EUR get 10 % and not 5 %?

3.4 Indentation Is Part of the Syntax

Python

```
if is_student:
    discount = 0.10
    print("Student discount")
print("Done")
```

Blocks are marked by a colon and indentation.

Java

```
if (isStudent) {
    discount = 0.10;
    System.out.println("Student discount");
}
System.out.println("Done");
```

Blocks are marked by braces { }; indentation only helps human readers.

- A colon : opens a block; all its lines are indented by the same amount (PEP 8: 4 spaces).
- The block ends where the indentation ends: print("Done") always runs.

```
IndentationError: expected an indented block after 'if' statement on line 1
```

Your turn!

Task

Predict what the code on the slide „Branching with if, elif and else“ prints for a) a customer who is not a student with 25.00 EUR, b) a student with 25.00 EUR, c) a customer who is not a student with 4.00 EUR. Then check in Colab.

Task

Extend your receipt program from 3.3: students get 10 % off, orders of 20 EUR or more get 5 % off. Print a discount line only if there is a discount.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.2 Functions

- From math we know that functions are a kind of assignment rules like:

$$f(x) = x + 2x$$

name, parameters, rule

- In computer science, we use functions to compute an output based on an input, like:

```
>>> max(1,2,3,4,5)
5
```



Functions without return values are termed “procedures”. But compared to other programming values (e.g. C or PASCAL) this makes no difference.

Adapted from Ernesti, J. & Kaiser, P. (2017); Offical Python Documentation (2019): https://docs.python.org

3.5 Functions Package a Calculation



Function
A named, reusable block of code. It takes inputs (**parameters**), runs its body and hands back an output (**return value**).

Why Functions?

- Write a calculation once and use it many times: don't repeat yourself.
- A good name explains the code: line_total(1.10, 3)
- Small functions are easy to test and to fix.
- Like a recipe: ingredients in, dish out – as often as you like.

Built-in Functions You Know

```
print()   input()   type()
int()     float()   str()
len()     round()   id()
```

Python ships with many built-in functions. Now you will write your own with **def**.

Adapted from Ernesti, J. & Kaiser, P. (2017); Offical Python Documentation (2019): https://docs.python.org

3.5 Anatomy of a Function

Definition and Call

```
def line_total(price, quantity):                    1
    """Return the price of one receipt line."""     2
    return price * quantity                         3

espresso = line_total(1.10, 3)                      4
croissant = line_total(1.80, 1)
```

- **Parameters** are the names in the definition: price, quantity.
- **Arguments** are the values in the call: 1.10, 3.
- A call runs the body and is replaced by the return value.

Parts of a Function

1. Header: def, the function's name, the parameters in parentheses and a colon
2. Docstring: says what the function does (optional, but good style)
3. Body, indented: return ends the function and hands the result back
4. Call: the arguments fill the parameters; the call evaluates to the return value

3.5 Returning versus Printing a Value

print() Only Shows

```
>>> def show_total(subtotal):
...     print(subtotal * 0.9)
...
>>> result = show_total(5.10)
4.59
>>> print(result)
None
```

- The value appears on the screen, but the function hands nothing back: the result is None.

return Hands Back

```
>>> def discounted(subtotal):
...     return subtotal * 0.9
...
>>> result = discounted(5.10)
>>> print(f"{result:.2f}")
4.59
```

- The caller receives the value and can store it, compute with it or print it.



Rule of thumb: functions compute and **return** their result; **print** it once, at the end.

Adapted from Ernesti, J. & Kaiser, P. (2017); Offical Python Documentation (2019): https://docs.python.org

3.5 Default Values and Local Scope

Default Values

```
>>> def line_total(price, quantity=1):
...     return price * quantity
...
>>> line_total(1.80)
1.8
>>> line_total(1.10, 3)
3.3000000000000003
>>> line_total(quantity=2, price=2.40)
4.8
```

- A parameter with a default value can be left out.
- Keyword arguments name the parameter, so their order does not matter.

Local Scope

```
>>> def apply_discount(subtotal, is_student):
...     rate = 0.0
...     if is_student:
...         rate = 0.10
...     return subtotal * (1 - rate)
...
>>> apply_discount(5.10, True)
4.59
>>> rate
NameError: name 'rate' is not defined
```

- Variables created inside a function are **local**: they exist only while it runs.
- Pass values in as parameters and get results back with return.

3.5 Three Kinds of Errors

Syntax Error

- The code breaks Python's grammar. Nothing runs at all.

```
if is_student
    discount = 0.10

SyntaxError: expected ':'
```

Runtime Error

- The code is valid but fails while it runs: Python raises an exception.

```
"Total: " + 4.59

TypeError: can only
concatenate str (not
"float") to str
```

- Others: NameError, ValueError, ZeroDivisionError

Logic Error

- The code runs, but the result is wrong. Python cannot notice it.

```
total = subtotal * discount
print(total)

0.51
```

- Expected 4.59: the discount was charged instead of subtracted.



Check your results with a small example you can compute by hand: 5.10 − 0.51 = 4.59.

3.5 How to Read a Traceback

A Runtime Error

```
def receipt_line(item, total):
    return item + ": " + total + " EUR"

print(receipt_line("Espresso", 3.30))
```

```
Traceback (most recent call last):
  File "/content/receipt.py", line 4, in <module>                    3
    print(receipt_line("Espresso", 3.30))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/content/receipt.py", line 2, in receipt_line                2
    return item + ": " + total + " EUR"
           ~~~~~~~~~~~~^~~~~~~
TypeError: can only concatenate str (not "float") to str             1
```

Read It from the Bottom Up

1. Start with the last line: the error type and its message. A str and a float cannot be joined with +.
2. One block up: where it happened – line 2, inside receipt_line. The ~ and ^ marks point to the faulty part.
3. Further up: who called it – line 4. The most recent call comes last.

Fix: format the number inside an f-string.

```
return f"{item}: {total:.2f} EUR"
```

3.5 yield and return

- Python provides generator functions as concentient way to build iterators:

```
def firstn(n):
    num = 0
    while num < n:
        yield num
        num +=1

top5 = firstn(5)
```

- The previous generator stops the execution and yields its items instead of returning a list

```
>>>next(top5)
0
>>>next(top5)
1
```

Adapted from Ernesti, J. & Kaiser, P. (2017); Offical Python Documentation (2019): https://docs.python.org

Your turn!

Task

Write the functions line_total(price, quantity=1) and apply_discount(subtotal, is_student). Use them to print the complete receipt from the start of this lecture, ending with TOTAL 4.59 EUR.

Task

Bug hunt: the notebook cell „Bug hunt“ contains one syntax error, one runtime error and one logic error. Find and fix all three.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

Recap: What Does This Code Print?

Predict, Then Discuss

```
# 1
type(459 / 100)
```

```
# 2
"3" * 2
```

```
# 3
is_student = False
subtotal = 25.00
if is_student:
    print("10 %")
elif subtotal >= 20:
    print("5 %")
```

```
# 4
def f(x):
    print(x * 2)

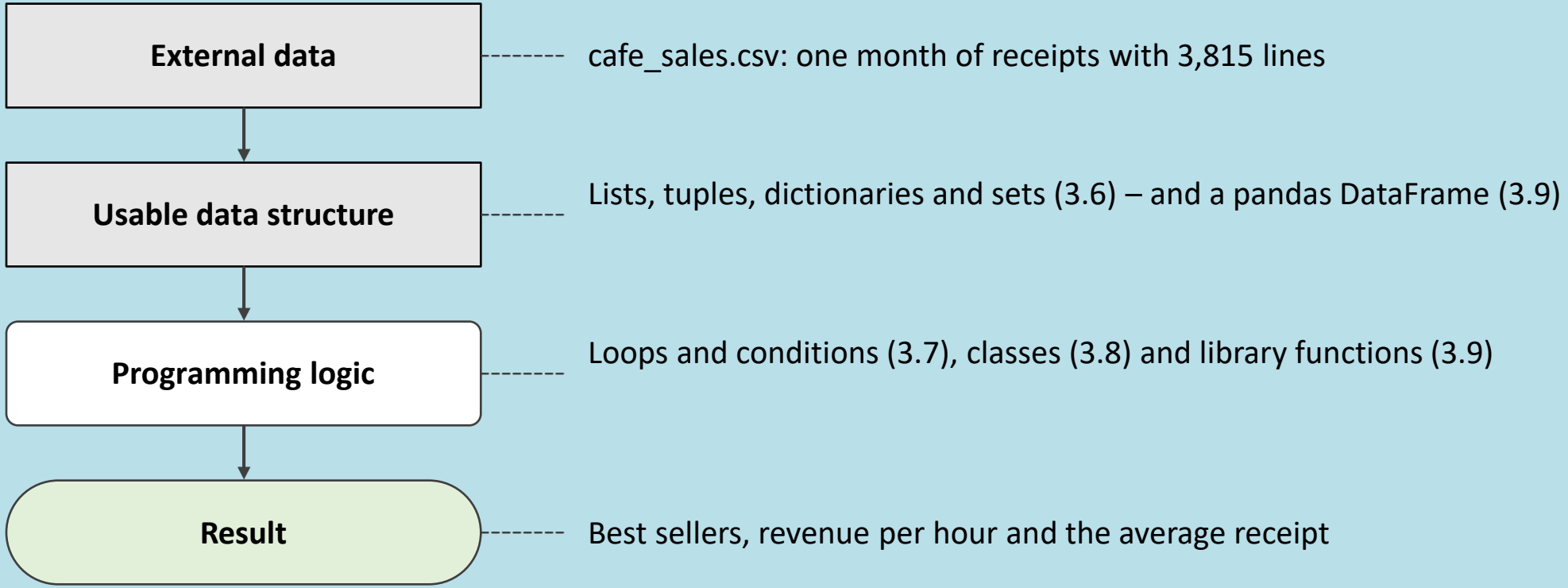
y = f(3)
print(y)
```



Discuss your predictions with your neighbour first. Then check them in Colab.

3.6 From Data to Logic

Typical Workflow, Applied to the Campus Café



Adapted from Ernesti, J. & Kaiser, P. (2017)

3.6 Lists: Ordered and Changeable

Creating and Reading

```
>>> order = ["Espresso", "Croissant", "Espresso"]
>>> order[0]
'Espresso'
>>> order[-1]
'Espresso'
>>> order[0:2]
['Espresso', 'Croissant']
>>> len(order)
3
```

- A **list** stores values in a fixed order; the index starts at 0.
- Negative indices count from the end; [start:stop] leaves out the stop element.

Changing a List

```
>>> order.append("Muffin")
>>> order.remove("Croissant")
>>> order
['Espresso', 'Espresso', 'Muffin']
>>> "Muffin" in order
True
>>> order.count("Espresso")
2
```

- Lists are **mutable**: you change them in place.
- append() changes the list and returns None.

3.6 Tuples: Fixed Records

Tuples

```
>>> line = ("Espresso", 1.10, 3)
>>> line[0]
'Espresso'
>>> item, price, quantity = line
>>> price * quantity
3.3000000000000003
>>> line[2] = 4
TypeError: 'tuple' object does not
support item assignment
```

- A **tuple** groups a fixed number of values into one record, e.g. one receipt line.
- Tuples are **immutable**: once created, they cannot change.
- Unpacking gives each value its own name.

A List of Tuples

```
>>> order = [("Espresso", 1.10, 3),
...          ("Croissant", 1.80, 1)]
>>> order[1]
('Croissant', 1.8, 1)
>>> order[1][0]
'Croissant'
```

- A list of tuples is a simple table: one tuple per row.
- Rule of thumb: a tuple for a fixed record whose positions have a meaning, a list for a collection that grows.

3.6 Dictionaries: Look Up by Key

Dictionaries

```
>>> menu = {"Espresso": 1.10,
...         "Croissant": 1.80,
...         "Cappuccino": 2.40}
>>> menu["Croissant"]
1.8
>>> menu["Tea"]
KeyError: 'Tea'
>>> menu.get("Tea", 0.0)
0.0
>>> menu["Tea"] = 1.50
>>> "Tea" in menu
True
>>> len(menu)
4
```

Key Ideas

- A **dictionary** maps keys to values – like a menu maps maps items to prices.
- You look up by key, not by position: menu["Croissant"].
- Keys are unique and immutable (str, int, tuple); values can be anything.
- A missing key raises a KeyError; .get() returns a default instead.
- Dictionaries keep the order in which keys were added.

3.6 Sets: Unique Values

Sets

```
>>> student_items = {"Espresso", "Muffin", "Espresso"}
>>> len(student_items)
2
>>> other_items = {"Espresso", "Pretzel"}
>>> sorted(student_items & other_items)
['Espresso']
>>> sorted(student_items | other_items)
['Espresso', 'Muffin', 'Pretzel']
>>> sorted(student_items - other_items)
['Muffin']
```

Which items did students buy, which did other customers buy, and which did which did both?

Key Ideas

- A **set** holds every value only once and has no order.
- Membership tests with in are very fast.
- & intersection, | union, - difference.
- sorted() shows a set in a fixed order.

3.6 Choosing the Right Data Structure

list, tuple, dict and set Compared

	list	tuple	dict	set
Example	["Espresso", "Muffin"]	("Espresso", 1.10, 3)	{"Espresso": 1.10}	{"Espresso", "Muffin"}
Ordered	yes	yes	yes (insertion order)	no
Changeable	yes	no	yes	yes
Duplicates	allowed	allowed	keys are unique	not possible
Access	by position	by position	by key	membership test (in)
Typical use	a collection that grows	a fixed record	looking up a value	unique values



A table is a list of records: each row a tuple or a dictionary. pandas builds on exactly this idea (3.9).

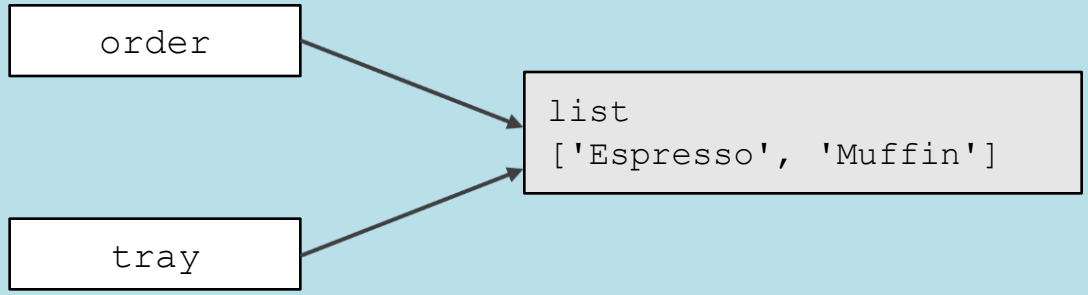
3.6 Shared References to Mutable Objects

Two Names, One List

```
>>> order = ["Espresso"]
>>> tray = order
>>> tray.append("Muffin")
>>> order
['Espresso', 'Muffin']
>>> tray is order
True
```

```
>>> tray = order.copy()
>>> tray.append("Tea")
>>> order
['Espresso', 'Muffin']
```

What Happens in Memory



Both names refer to the same list. A change through one name is visible visible through the other.

Mutable Object
An object that can change after it was created: list, dict, set. An assignment never copies it; use .copy() when you need an you need an independent copy.

Your turn!

Task

Store the menu of the Campus Café in a dictionary and an order of 3 espressos and 1 and 1 croissant as a list of tuples (item, quantity). Then print the price of a croissant. croissant.

Task

Which data structure fits best? (a) all receipt numbers of one day in order, (b) the price of every item, (c) the different items a customer has ever bought, (d) the opening hours of one day as (opens, closes). Explain your choice.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.7 for Loops: Repeat for Each Element

A for Loop

```
order = [("Espresso", 3), ("Croissant", 1)]
menu = {"Espresso": 1.10, "Croissant": 1.80,
        "Cappuccino": 2.40}

for item, quantity in order:
    price = menu[item]
    print(f"{quantity} x {item}: {price * quantity:.2f}")
```

```
3 x Espresso: 3.30
1 x Croissant: 1.80
```

- for takes one element after the other and runs the indented body for each.
- The loop variables item and quantity are unpacked from each tuple.

More Ways to Loop

```
>>> for i in range(3):
...     print(i)
0
1
2
>>> for item, price in menu.items():
...     print(item, price)
Espresso 1.1
Croissant 1.8
Cappuccino 2.4
```

- range(n) counts from 0 to n − 1.
- .items() gives the pairs of a dictionary.

3.7 Three Loop Patterns

Accumulate

```
total = 0
for item, quantity in order:
    total += menu[item] * quantity
print(round(total, 2))          # 5.1
```

Start with a neutral value and update it update it in every round. total += x is x is short for total = total + x.

Filter

```
coffees = []
for item, quantity in order:
    if item in ("Espresso", "Cappuccino"):
        coffees.append(item)
print(coffees)                  # ['Espresso']
```

Start with an empty list and keep only the elements that meet a condition.

Find the Maximum

```
best, best_price = None, 0
for item, price in menu.items():
    if price > best_price:
        best, best_price = item, price
print(best)                     # Cappuccino
```

Remember the best element so far and replace it whenever a better one turns up.

3.7 while Loops, break and continue

A Cash Register

```
total = 0
while True:
    item = input("Item (or 'done'): ")
    if item == "done":
        break
    if item not in menu:
        print("Unknown item")
        continue
    total += menu[item]

print(f"Total: {total:.2f} EUR")
```

```
Item (or 'done'): Espresso
Item (or 'done'): Tea
Unknown item
Item (or 'done'): Croissant
Item (or 'done'): done
Total: 2.90 EUR
```

Key Ideas

- while repeats its body as long as the condition condition is True – use it when you do not not know in advance how often.
- break leaves the loop at once; continue jumps to the next round.
- Make sure the loop can end. In Colab, stop a runaway cell with its stop button.

3.7 Handling Errors with try and except

Asking Until the Input Is Valid

```
while True:
    try:
        quantity = int(input("Quantity: "))
        break
    except ValueError:
        print("Please enter a whole number.")
```

```
Quantity: three
Please enter a whole number.
Quantity: 3
```

Key Ideas

- Code that might fail goes into **try**; **except** catches a specific exception and handles it.
- Catch only the exceptions you expect. A bare except also hides real bugs.



Remember Task 3.3? int("three") raised a ValueError. Now your program can handle it – „Errors should never pass silently.“ (Zen of Python)

3.7 From Pseudocode to Python

Pseudocode as in the Textbook

```
function RECEIPT-TOTAL(order, menu) returns a number
  total ← 0
  for each (item, quantity) in order do
    total ← total + menu[item] × quantity
  if total ≥ 20 then
    total ← total × 0.95
  return total
```

Python

```
def receipt_total(order, menu):
    total = 0
    for item, quantity in order:
        total = total + menu[item] * quantity
    if total >= 20:
        total = total * 0.95
    return total
```

Translation

	Assignment	Loop	Condition	Function	Blocks
Pseudocode	total ← 0	for each x in s do	if c then	function F(a) returns …	indentation
Python	total = 0	for x in s:	if c:	def f(a):	colon + indentation

Pseudocode conventions as in Russell, S. & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.)

Your turn!

Task

Write the cash register loop: ask for items until the user types done, report unknown items unknown items and print the total.

Task

Translate this pseudocode into Python and test it with [("Espresso", 3), ("Croissant", 1), ("Espresso", 2)]:

```
function COUNT-ITEMS(order) returns a dictionary
  counts ← an empty dictionary
  for each (item, quantity) in order do
    if item is a key of counts then
      counts[item] ← counts[item] + quantity
    else counts[item] ← quantity
  return counts
```

Outline

3	AI Programming with Python
3.1	Python in a Nutshell
3.2	Values, Types and Variables
3.3	Input and Output
3.4	Control Structures
3.5	Functions and Errors
3.6	Data Structures
3.7	Loops
3.8	Classes and Objects
3.9	Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



▶ **What we will learn in Part 2:**

- How to store many values in lists, tuples, dictionaries and sets
- How to repeat steps with loops and translate pseudocode into Python
- How to write classes and analyse data with NumPy, pandas and matplotlib

▶ **Duration:**

- Part 1 (3.1 – 3.5): 90 min
- Part 2 (3.6 – 3.9): 90 min

▶ **Relevant for Exam:**

- 3.1 – 3.9

3.8 Classes Bundle Data and Behaviour



Class
A blueprint for objects. It defines which data each object stores (**attributes**) and what it can do (**methods**). An object created from a class is an **instance** of that class.

You Already Use Objects

```
>>> item = "Espresso"
>>> item.upper()
'ESPRESSO'
>>> order = []
>>> order.append(item)
>>> type(order)
<class 'list'>
```

str and list are classes; upper() and append() are their methods.

Why Write Your Own Classes?

- Keep data and the functions that work on it together.
- Create many independent objects from one blueprint: one receipt per customer.
- Read other people's code: many libraries – and the AI code later in this course – are built from classes.

3.8 Defining a Class

The Receipt Class

```
class Receipt:                                                    1
    """A receipt of the Campus Café."""

    def __init__(self, is_student=False):                         2
        self.lines = []                                           3
        self.is_student = is_student

    def add(self, item, price, quantity=1):                       4
        self.lines.append((item, price, quantity))

    def total(self):
        subtotal = 0
        for item, price, quantity in self.lines:
            subtotal += price * quantity
        if self.is_student:
            subtotal = subtotal * 0.9
        return round(subtotal, 2)
```

Parts of a Class

1. class Receipt: starts the blueprint. Class names use CapWords.
2. __init__ is the constructor: it runs when a new object is created.
3. self.lines and self.is_student are attributes: the data each object stores.
4. add() and total() are methods. Their first parameter, self, is the object itself.

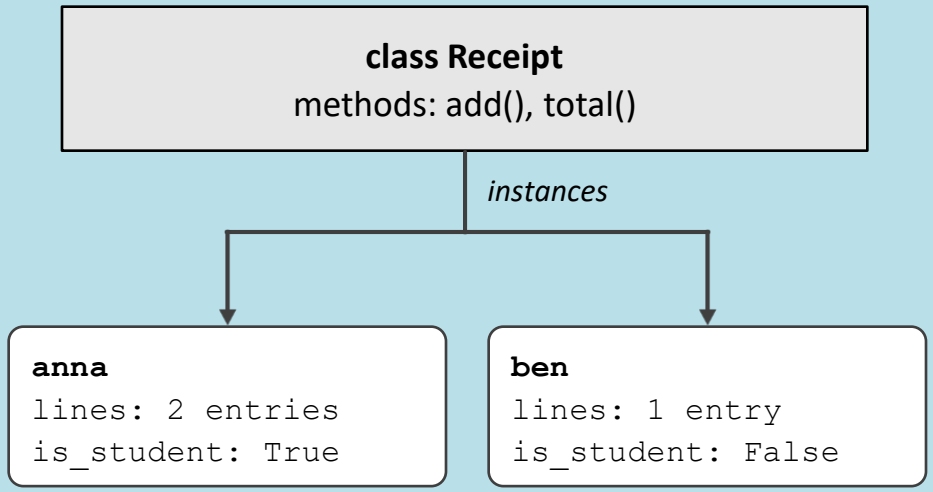
3.8 Creating and Using Objects

Two Receipts

```
>>> anna = Receipt(is_student=True)
>>> anna.add("Espresso", 1.10, 3)
>>> anna.add("Croissant", 1.80)
>>> anna.total()
4.59
>>> ben = Receipt()
>>> ben.add("Cappuccino", 2.40)
>>> ben.total()
2.4
>>> len(anna.lines), len(ben.lines)
(2, 1)
```

- Calling the class creates a new object and runs __init__.
- Each object keeps its own attributes; a method works on the object it is called on (self).

One Class, Two Objects



Same methods, separate data: each receipt has its own lines. own lines.

Your turn!

Task

Add a method item_count(self) to Receipt that returns the number of items on a receipt, i.e. receipt, i.e. the sum of all quantities. For anna it should return 4.

Task

Add a method print_receipt(self) that prints the receipt from Part 1. Reuse your code from Task 3.5a.

Outline

3 AI Programming with Python

- 3.1 Python in a Nutshell
- 3.2 Values, Types and Variables
- 3.3 Input and Output
- 3.4 Control Structures
- 3.5 Functions and Errors
- 3.6 Data Structures
- 3.7 Loops
- 3.8 Classes and Objects
- 3.9 Modules and Libraries

Lectorial 1: Implement Problem-Solving Agents with Python



Image source: ↗ Pixabay (2019) / ↗ CC0

► **Duration:**
- 90 min

► **Relevant for Exam:**
- 3.1 - 3.4

3.9 Modules

- We learnt that in Python exists many build-in functions you can use

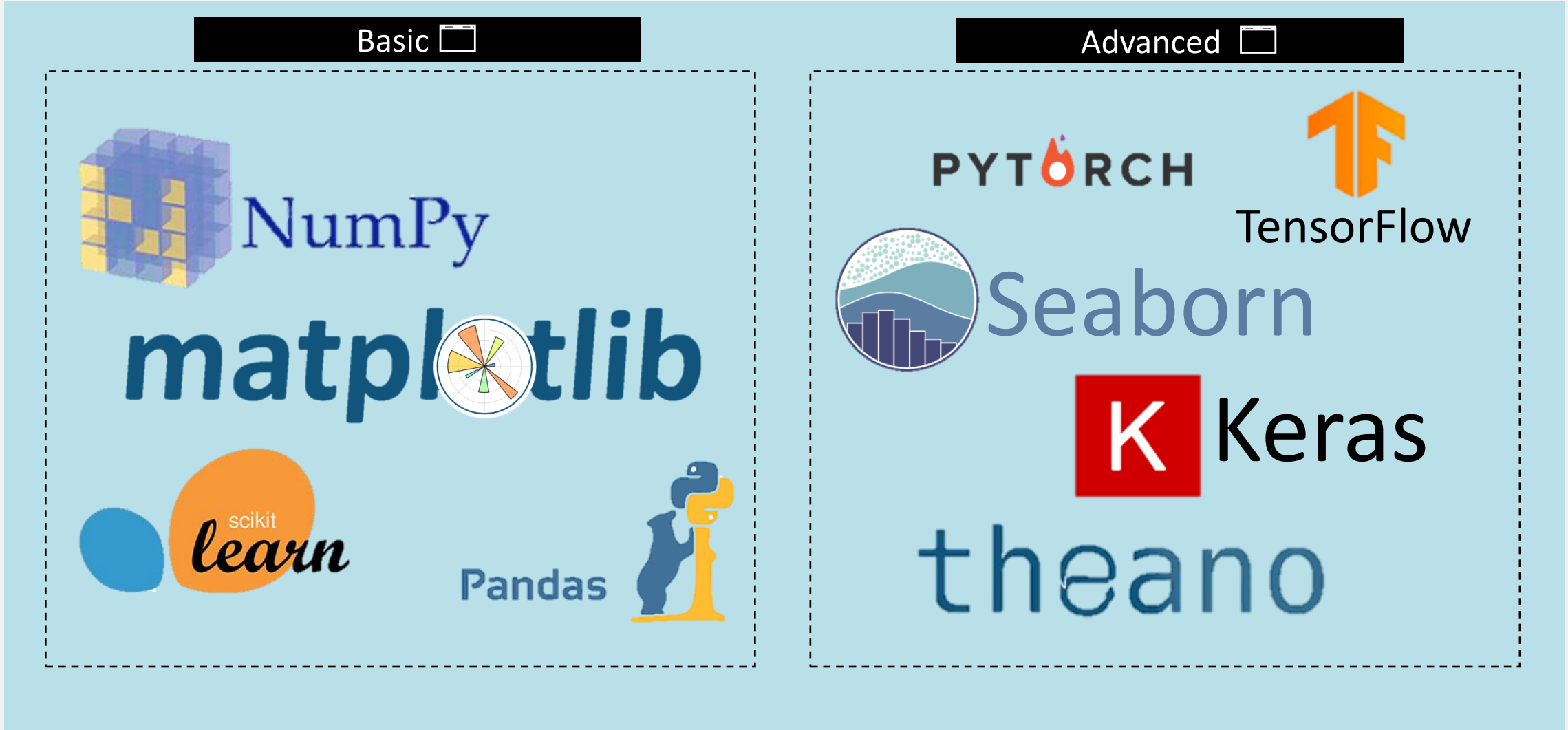
```
>>> max([1,5,2,7,42,3])
42
```

build-in functions

- However, there are many functions organized in external modules (like specific AI functions) we can use in our programm

Adapted from Ernesti, J. & Kaiser, P. (2017)

3.9 Most relevant Modules for AI-Specialists



Images: From the respective websites

3.9 Modules Import

- Import moduls

```
import module
module.spec_func()
```

- Import specific function from modul

```
from module import spec_func
spec_func()
```

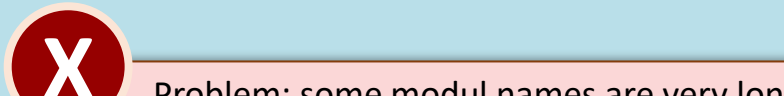
- Import moduls with specific namespaces

```
import module as mo
mo.spec_func()
```

- Import every function from modul

```
from moduls import *
spec_func()
```





Problem: some modul names are very long and you import functions you probably do not need

import os



from os import *





Do this only if you know what you are doing!

Offical Python Documentation (2019): https://docs.python.org

3.9 Use Modules in Python

- You can import existing modules (global modules) with `import`

```
>>> import math
>>> math.sin(math.pi)
1.2246467991473532e-16
```

```
>>> from math import *
>>> sin(math.pi)
1.2246467991473532e-16
```

```
>>> from math import sin, pi
>>> sin(math.pi)
1.2246467991473532e-16
```

Adapted from Ernesti, J. & Kaiser, P. (2017); Offical Python Documentation (2019): https://docs.python.org

3.9 Modules, Packages and Libraries

Terms

Term	Meaning	Examples
Module	a Python file with functions, classes and variables	math, random
Package	a folder of modules that belong together	pandas, matplotlib
Standard library	modules that come with every Python installation	math, random, datetime, csv
Third-party package	installed from PyPI with pip; Colab already has the common ones	numpy, pandas, matplotlib
Virtual environment	a separate set of packages for one project	venv, conda

```
import math                 # math.sqrt(16)
from math import sqrt       # sqrt(16)
import pandas as pd         # pd.read_csv(...)
%pip install seaborn        # install a package in Colab
```



Avoid from module import *: you no longer see where a where a name comes from, and names can silently overwrite each other.

3.9 NumPy: Calculating with Whole Arrays

Arrays

```
>>> import numpy as np
>>> prices = np.array([1.10, 1.80, 2.40])
>>> quantities = np.array([3, 1, 2])
>>> prices * quantities
array([3.3, 1.8, 4.8])
>>> print((prices * quantities).sum())
9.9
>>> prices > 1.5
array([False,  True,  True])
>>> prices[prices > 1.5]
array([1.8, 2.4])
```



Without print(), a notebook shows such a sum as np.float64(9.9): the same number, with its NumPy type.

Key Ideas



- A NumPy **array** holds many numbers of the the same type.
- Operations apply to all elements at once (**vectorization**): no loop needed, and much faster on large data.
- A comparison returns an array of True and False that selects elements.

3.9 pandas: Tables in Python

Loading a CSV File

```
>>> import pandas as pd
>>> sales = pd.read_csv("cafe_sales.csv", sep=";", decimal=",",
...                     parse_dates=["timestamp"])
>>> sales.shape
(3815, 7)
```

- A **DataFrame** is a table: rows are records, columns have a name and a type.
- sep and decimal read German CSV files with ; and decimal commas.

sales.head(3)

	receipt_id	timestamp	item	category	quantity	unit_price	student
0	10001	2025-10-01 07:38:00	Tea	drink	1	1.5	False
1	10001	2025-10-01 07:38:00	Cappuccino	coffee	1	2.4	False
2	10001	2025-10-01 07:38:00	Espresso	coffee	1	1.1	False

3.9 Asking Questions with pandas

Code

```
sales["revenue"] = sales["quantity"] * sales["unit_price"]
sales["hour"] = sales["timestamp"].dt.hour

top = (sales.groupby("item")["quantity"].sum()
           .sort_values(ascending=False))
print(top.head(3))

food = sales[sales["category"] == "food"]
print(round(food["revenue"].sum(), 2))

per_receipt = sales.groupby("receipt_id")["revenue"].sum()
print(round(per_receipt.mean(), 2))
```

- New columns are computed for all rows at once.
- A condition in brackets filters the rows.
- groupby splits the table into groups and aggregates each group: sum, mean, count.

Output

Which items sell best?

```
item
Cappuccino    682
Espresso      511
Sandwich      487
Name: quantity, dtype: int64
```

Revenue from food (EUR)

```
3720.1
```

Average receipt (EUR)

```
3.54
```

3.9 matplotlib: Revenue per Hour

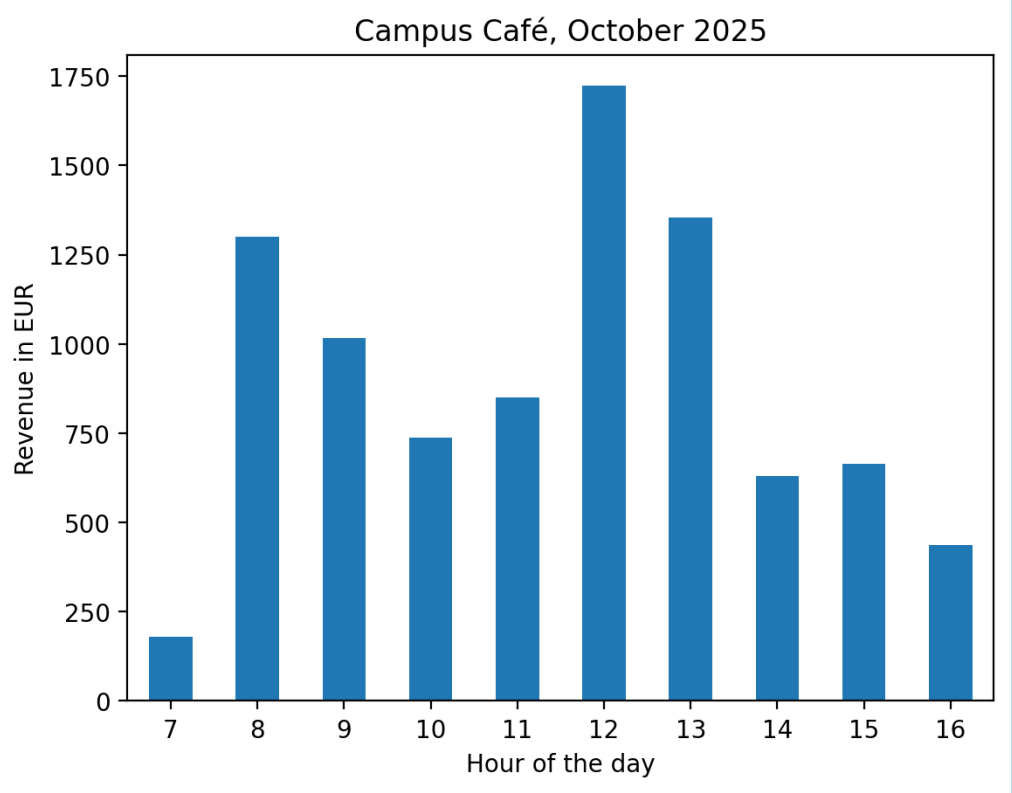
Code

```
import matplotlib.pyplot as plt

per_hour = sales.groupby("hour")["revenue"].sum()
per_hour.plot(kind="bar", rot=0)
plt.xlabel("Hour of the day")
plt.ylabel("Revenue in EUR")
plt.title("Campus Café, October 2025")
plt.show()
```

- plot() draws a pandas Series or DataFrame with matplotlib.
- Always label the axes and give the chart a title.
- kind="bar" for categories, kind="line" for time series.

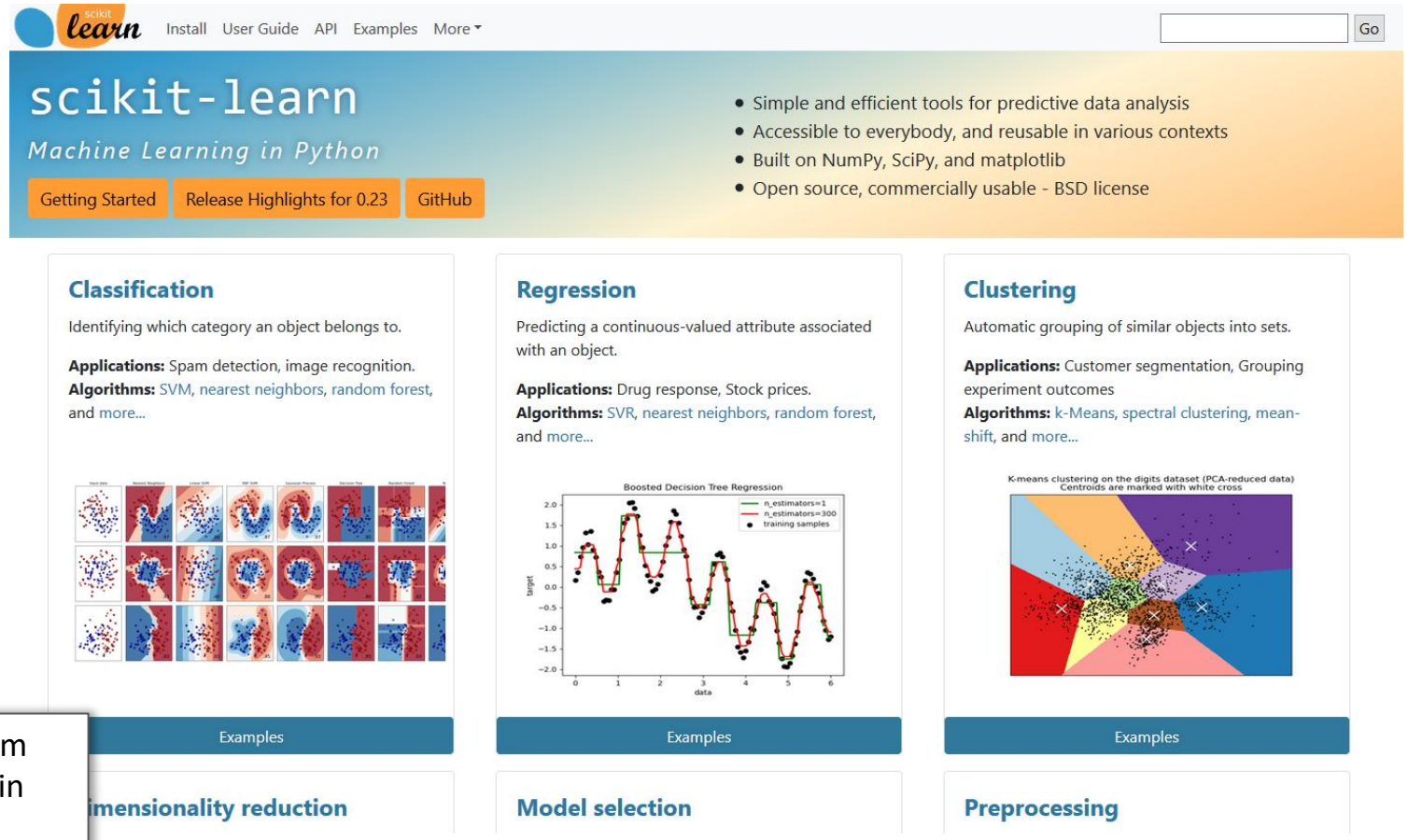
Result



Two peaks: before the first lecture (8–9) and at lunch (12–13).

3.9 Scikit-learn

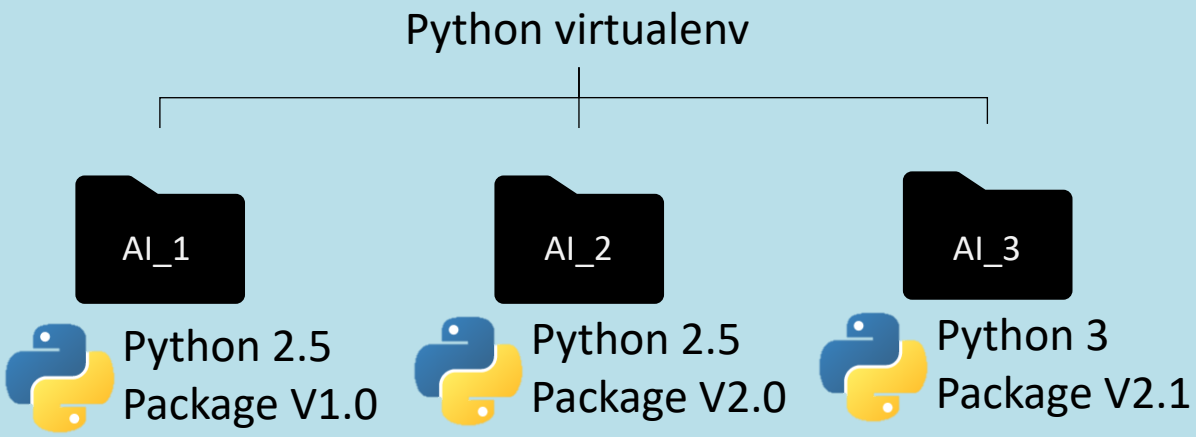
```
>>> import sklearn
```



We will use the AI algorithms from this package (see e.g. chapter 5,6) in the following parts of this lecture!

3.9 Management of Different Packages and Versions?

- Python virtual environments allow to create an isolated environment for Python projects so that each of your projects has its own individual dependencies, regardless of what dependencies every other project has.



Offical Python Documentation (2019): https://docs.python.org

Your turn!

Task

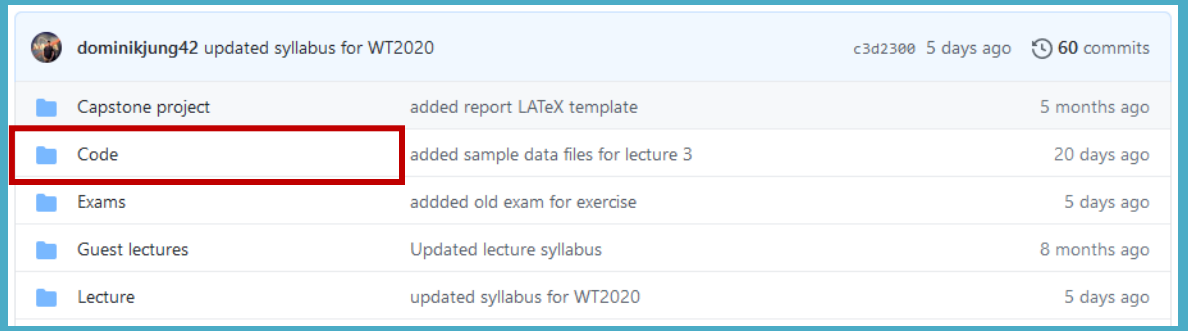
Load cafe_sales.csv in Colab and answer with pandas: Which item sells best? Which hour Which hour brings the most revenue? What is the average value of a receipt?

Task

Bonus: Plot the revenue per weekday. On one weekday in October 2025 the café sold nothing at all. Which day was it, and why?

Your first end-to-end project: Problem-Solving Agents

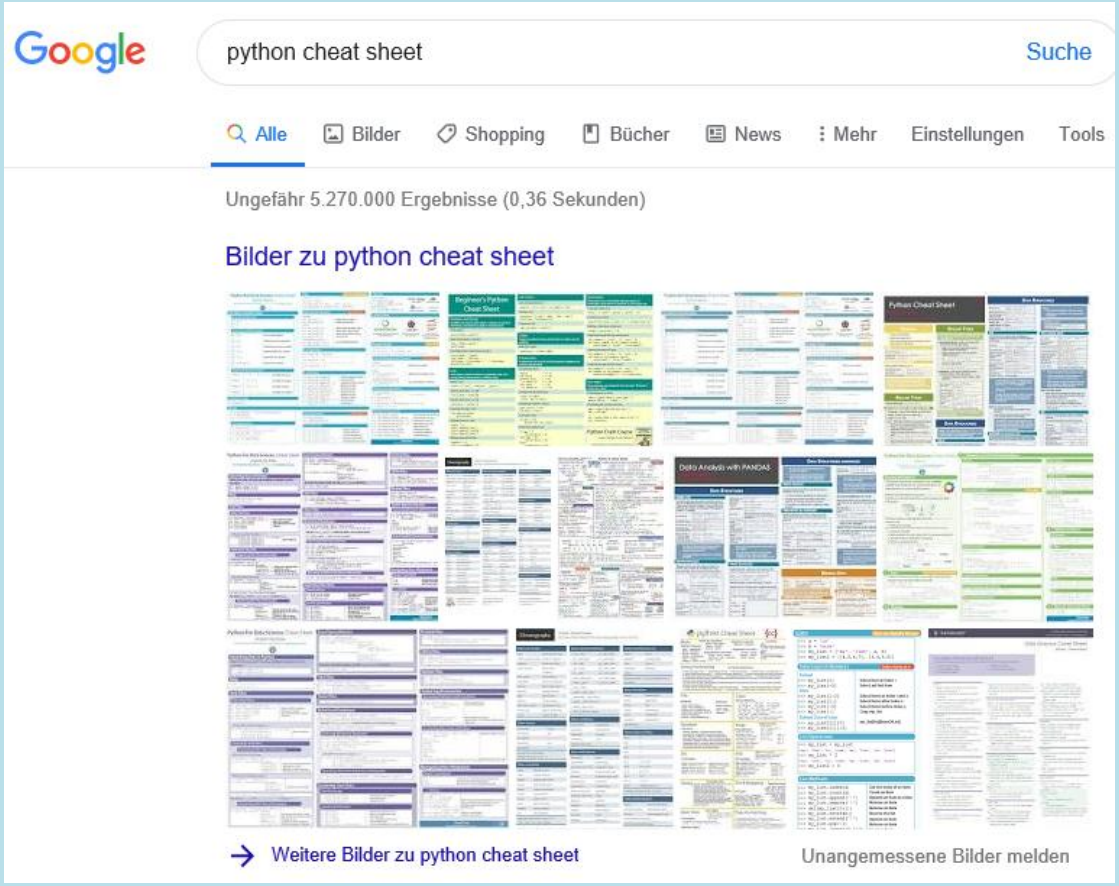




Feel free to start directly or just continue with the lecture and come back to it later.

Handouts

Python Cheatsheet



- Python cheatsheets are very helpful to learn the basics
- They exist for very different topics

Exam-Style Questions

Test Yourself

1. Explain the difference between syntax and semantics with one Python example each.
2. What is the difference between a compiled and an interpreted language? Where does Python fit in?
3. Python is dynamically and strongly typed. Explain both terms with an example.
4. Why does 0.1 + 0.2 == 0.3 evaluate to False? How should you compare floats instead?
5. What does the code on the right print for is_student = True and subtotal = 25.00? Why?
6. What is the difference between a parameter and an argument, and between print() and return?

Code for Question 5

```
if is_student:
    discount = 0.10
elif subtotal >= 20:
    discount = 0.05
else:
    discount = 0.0

total = subtotal * (1 - discount)
print(f"Total: {total:.2f} EUR")
```

Exam-Style Questions

Test Yourself

1. Compare list, tuple, dict and set: order, mutability, duplicates, access. Name one use case each.
2. What does the code on the right print, and why?
3. Name three loop patterns and write the Python skeleton of one of them.
4. When do you use while instead of for? What do break and continue do?
5. What is the difference between a class and an object? What does self refer to?
6. What does vectorization mean in NumPy, and why is it faster than a Python loop?

Code for Question 2

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Exercises

Coding Exercises

- Work through the notebook 03a_Python_Foundations.ipynb in Google Colab: all tasks of 3.1 – 3.5.
- Compare your code with the solution notebook 03a_Python_Foundations_Solutions.ipynb.
- Answer the exam-style questions in your own words.

Optional: The Python Tutorial, chapters 3 and 4. ↗ https://docs.python.org/3/tutorial/ – self-study.

You do not have to finish the exercises right after the lecture. But they will help you a lot in Lectorial 1.

Exercises



Coding Exercises

- Work through the notebook 03b_Python_Data.ipynb in Google Colab, using the data set cafe_sales.csv.
- Compare your code with the solution notebook 03b_Python_Data_Solutions.ipynb.
- Answer the exam-style questions in your own words.



Optional: The Python Tutorial, chapters 5 and 9 S https://docs.python.org/3/tutorial/ and „10 minutes to pandas“ S S https://pandas.pydata.org/docs/user_guide/10min.html – self-study. study.



You do not have to finish the exercises right after the lecture. But they But they will help you a lot in Lectorial 1.

▶ **Next:** Lectorial 1 – Implement Problem-Solving Agents with Python.

References

Literature

1. Ernesti, J., & Kaiser, P. (2017). *Python 3: das umfassende Handbuch*. Rheinwerk Verlag.
2. Offical Python Documentation (2020). The Python Language Reference. Online available at https://docs.python.org

News articles

1. Waggoner, P (2018): Advice to Young (and Old) Programmers: A Conversation with Hadley Wickham. Online available at https://www.r-bloggers.com/2018/08/advice-to-young-and-old-programmers-a-conversation-with-hadley-wickham/

References

Images

All images that were not marked other ways are made by myself, or licensed ↗CC0 from ↗Pixabay.

Further reading

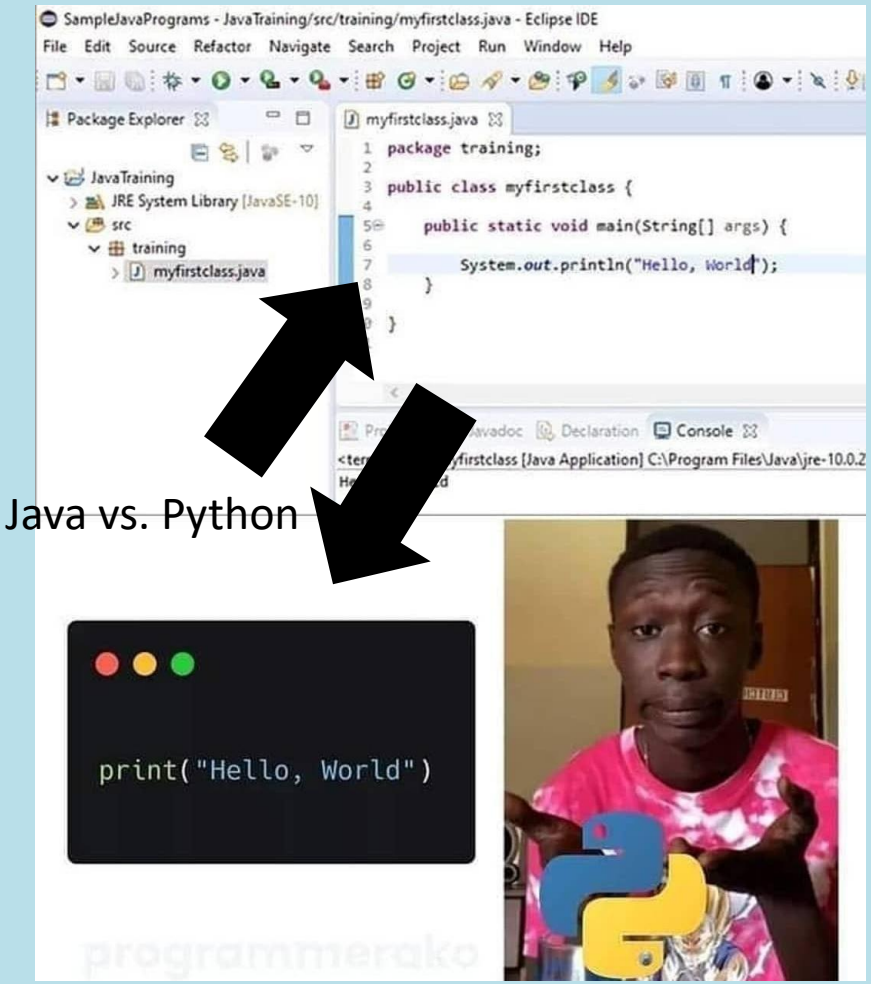
- I also can recommend to take a look at the beginner tutorials of the different Python packages (↗Numpy, ↗Matplotlib, ↗Scikit-learn, ↗Pandas)
- Python has a manifest of ist design principles, which you can find inside the Python interpreter itself by typing `import this`. Check it out!

Glossary

Data Frame	*A two-dimensional data structure, where data is aligned in a tabular fashion in rows and columns.*
IDE	*Integrated Development Environment. Applications that facilitate the development of software or code by giving the user an interface to work with.*
Libraries/Packages	*Collection of pre-written code (functions and methods) to perform certain task.*
Notebooks	*Virtual environment that enables literate programming.*
Programming language	*Language engineered to create a standard form of instructions for a computer. Like human languages they are split into two components, i.e. syntax (form) and semantics (meaning).*
Software Tool	*Software that is designed to be used for a specific use case (e.g. Dashboarding, Data Visualization, Modelling).*

The Zen of Python (Tim Peters)

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!



Offical Python Documentation (2019): https://docs.python.org